

Halo 2 Guide

The Halo 2 proof system, from arithmetisation to recursion

m@rek.onl

Abstract

Assuming the mathematics of the *Math Guide* and the cryptographic primitives of the *Crypto Guide*, this volume of *The Zcash Arboretum* develops the Halo 2 proof system: the polynomial-IOP design pattern, a rigorous account of knowledge soundness and the algebraic group model, recursive proof composition and accumulation schemes, and finally Halo 2 itself — PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment, the Halo accumulation trick, and the complete protocol. A closing worked example carries one concrete statement from computation to verifier acceptance, with every number computed in $\mathbb{F}_{97}$.

Contents

1	Introduction	3
2	Polynomial IOPs and the SNARK design pattern	3
2.1	The polynomial IOP, formally	3
2.2	Compiling a PIOP into a SNARK	3
2.3	A worked example: a PIOP for univariate evaluation	4
2.4	PLONK as a PIOP	5
3	A rigorous treatment of knowledge soundness	5
3.1	Definitional refinements	5
3.2	The algebraic group model	6
3.3	Tree-extractor analysis	6
3.4	Concrete versus asymptotic security	7
4	Recursive proof composition and accumulation schemes	7
4.1	Recursion and its two classical obstacles	7
4.2	Accumulation schemes	7
4.3	A cycle of curves for efficient native recursion	8
5	The Halo 2 proof system	8
5.1	PLONKish arithmetisation	8
5.2	From statement to circuit: arithmetisation in practice	9
5.3	The permutation (equality) argument	13
5.4	The lookup argument	14
5.5	The inner-product polynomial commitment	14
5.5.1	The recursive halving protocol	15
5.5.2	The structured scalars and the polynomial g	15
5.5.3	Worked example: an IPA opening at $d = 4$	16
5.5.4	Knowledge soundness via rewinding	16
5.6	Accumulation and the Halo trick	17
5.7	The complete Halo 2 protocol	18
5.8	Instance columns and binding the public statement	20
5.9	Zero knowledge: hiding the witness in Halo 2	21
6	Worked example: one computation, carried from claim to conviction	23
6.1	What we are going to prove, and what “prove” means	23
6.2	From computation to a grid of constraints	24
6.3	From the grid to polynomials	25
6.4	From vanishing to a single identity	25
6.5	Why checking one random point is a proof	26
6.6	Why the prover cannot wriggle: binding commitments	27
6.7	Enforcing the wiring: the permutation argument	28
6.8	Proving a value sits in a table: lookups	29
6.9	Binding the public statement, and hiding the witness	30
6.10	Removing the verifier: the Fiat–Shamir transform	30
6.11	The “Halo” in Halo 2: deferring the expensive check	31
6.12	Why the verifier is convinced	31

1 Introduction

This is the *Halo 2 Guide*, the third volume of *The Zcash Arboretum*, developing the cryptography behind Zcash’s Orchard protocol. The first two volumes — the *Math Guide* and the *Crypto Guide* — constructed, from the ground up, the mathematics (finite fields, elliptic curves, polynomials and evaluation domains, probability) and the cryptographic primitives (hash functions, commitments, Σ -protocols and the Fiat–Shamir transform, zero knowledge, polynomial commitment schemes) that this volume assumes. The present volume assembles those ingredients into a *proof system*.

The formal development proceeds in four steps. First, we make the polynomial-IOP design pattern precise: the recipe “encode a computation as polynomial identities over an evaluation domain, then compile to a succinct argument with a polynomial commitment.” Next, we treat knowledge soundness rigorously, including the algebraic group model and the tree-of-transcripts extractor that the inner-product argument requires. We then motivate recursive proof composition and develop the accumulation schemes that make it practical. Finally, we assemble Halo 2 itself: PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment with a worked example and its accumulation (the “Halo trick”), and the complete seven-phase protocol. Orchard uses this proof system to construct its shielded payment relation.

A reader who wants the concrete picture before the formal machinery may begin with Section 6. It carries the claim “I know x with $x^3 + x + 5 = 35$ ” through a grid of constraints, polynomials, a random-point check, commitments, wiring, lookups, Fiat–Shamir, and accumulation. Every number is small enough to verify by hand; Sections 2– 5 then supply the definitions and security arguments that the example deliberately sketches.

2 Polynomial IOPs and the SNARK design pattern

The modern SNARK design pattern factors the protocol into two clean layers: a *polynomial interactive oracle proof* (PIOP) for the relation, and a polynomial commitment scheme (the *Crypto Guide*) that realises the PIOP’s oracles. This factoring, due in its current form to Bünz, Fisch, Szepieniec, and others, has substantially clarified the nature of SNARKs.

2.1 The polynomial IOP, formally

Definition 2.1 (Polynomial IOP). A *polynomial interactive oracle proof* for a relation $\mathcal{R}$ is a public-coin interactive protocol where in each round:

- The prover sends one or more *polynomial oracles* $p_i \in \mathbb{F}[X]$ (each of bounded degree).
- The verifier sends a random challenge $c \in \mathbb{F}$.

At the end of the protocol, the verifier may query each oracle at a small number of points, learning the polynomial’s evaluations at those points. The verifier’s final accept/reject is a deterministic function of the challenges, evaluations, and the public input x .

We define completeness, soundness, and knowledge soundness as usual.

A PIOP differs from a vanilla interactive argument in one essential respect: the verifier does not read the prover’s messages in full — it queries only a few evaluations of each polynomial oracle. This is what renders the verifier “succinct” even when the polynomials themselves have large degree.

2.2 Compiling a PIOP into a SNARK

Construction 2.2 (PIOP-to-argument compiler). Given a PIOP Π for $\mathcal{R}$, a PCS scheme (Setup, Commit, Open, Verify) for $\mathbb{F}[X]$, and a random oracle H , the compiler produces a non-interactive argument:

1. Setup: run the PCS setup to obtain pp .
2. Prover: simulate Π , replacing each polynomial-oracle message p_i by its PCS commitment C_i . Use Fiat–Shamir (the *Crypto Guide*) to derive verifier challenges from the transcript, and supply PCS openings for the PIOP’s oracle queries.
3. Verifier: parse the transcript, recompute Fiat–Shamir challenges, verify each PCS opening, and run the PIOP’s final accept/reject check.

A PIOP is *round-by-round* knowledge sound, informally, when partial transcripts admit a “doomed” labelling that a prover message cannot escape except with bounded probability over the next verifier challenge. A related *state-restoration* definition lets a prover restore an earlier verifier state and redraw its challenge. Precise equivalence or implication theorems between these notions require additional protocol and extractor conditions; we do not identify them definitionally here.

Theorem 2.3 (Compilation principle, informal). *Suppose Π has a round-by-round or state-restoration knowledge-soundness theorem, the PCS supplies compatible online extraction and binding properties, and the transcript satisfies the hypotheses of an appropriate multi-round Fiat–Shamir theorem. Then the compiled protocol is a non-interactive argument of knowledge in the random-oracle model. The exact error and query loss depend on those definitions and on the extraction model; they are not determined solely by two scalar errors $\kappa_\Pi, \kappa_{\text{PCS}}$.*

Fiat–Shamir exposes a query-budget loss because changing a commitment redraws the later challenges. Plain nested forking can lose $Q^{O(r)}$ for r rounds; stronger round-by-round or state-restoration notions are designed to obtain tighter bounds. Claiming a single factor Q nevertheless requires the precise theorem’s state-restoration, transcript, challenge, and extraction hypotheses.

This factoring is of considerable utility: research on PIOPs (PLONK, Marlin, Spartan, HyperPlonk, ...) proceeds independently of research on polynomial commitment schemes, and any compatible pair combines into a new SNARK that inherits the PIOP’s relation and the commitment scheme’s cryptographic profile. This document develops the one PCS that Halo 2 uses — the inner-product argument (§5.5) — and treats the PIOP/PCS split only insofar as it clarifies the Halo 2 construction.

2.3 A worked example: a PIOP for univariate evaluation

To render the abstraction concrete, consider a minimal PIOP for the simplest non-trivial relation: “the prover knows a polynomial p of degree $\leq d$ such that $p(z) = y$ for a public (z, y) .”

Construction 2.4 (PIOP for univariate evaluation). On input (z, y) and prover witness p :

1. Prover sends the degree- $\leq d$ polynomial oracle p and the degree- $\leq d - 1$ quotient oracle $q := (p - y)/(X - z)$.
2. Verifier samples a random challenge $r \in \mathbb{F}$.
3. Verifier queries p and q at r .
4. Verifier accepts iff $p(r) - y = q(r) \cdot (r - z)$.

Completeness is immediate (q is a polynomial since $p(z) - y = 0$). Soundness follows from

Schwartz–Zippel: if $(X - z) \nmid (p(X) - y)$, then $p(X) - y - q(X)(X - z)$ is a non-zero polynomial of degree $\leq d$, and its random evaluation is zero with probability $\leq d/|\mathbb{F}|$.

Compiling this PIOP with the IPA-PCS and applying Fiat–Shamir yields a non-interactive argument for the same relation. Its opening proof is logarithmic in the degree bound, but the final IPA MSM makes verification linear unless it is deferred by an accumulation scheme. The PCS handles polynomial hiding and binding; the PIOP handles the relation logic. This separation is precisely the modular proof-system design pattern.

2.4 PLONK as a PIOP

PLONK (§5.1) is a more elaborate PIOP. Its oracles are:

- One advice polynomial per advice column.
- A permutation running-product polynomial Z_{perm} .
- Two permuted lookup columns A', S' per lookup.
- One lookup running-product polynomial per lookup.
- A quotient polynomial h (split into chunks).

Its challenges are, in order: θ (lookup column compression), β, γ (the permutation and lookup products), y (the gate-equation combination), and x (the final evaluation point). Its final check is the PLONK gate equation $C(x) = h(x) \cdot Z_H(x)$, which the verifier checks from the evaluations of the oracles at x (and possibly ωx).

This is precisely the PIOP that phases 1–5 of the Halo 2 protocol of §5.7 compile, phase 5’s claimed evaluations answering its oracle queries; phases 6–7 bind those answers to the commitments — the multipoint reduction (the challenges $x_1, \dots, x_4$ and the combined polynomial f^*) followed by a single IPA-PCS opening of f^* .

3 A rigorous treatment of knowledge soundness

We now develop knowledge soundness with the care needed for arguments about modern SNARKs. The treatment combines the rewinding-extractor framework (the *Crypto Guide*) with the algebraic group model.

3.1 Definitional refinements

The bare definition of the *Crypto Guide* suffices for Σ -protocols but is awkward for compound systems. We record the following standard refinement.

Remark 3.1 (Witness-extended emulation). An interactive argument $(\mathcal{P}, \mathcal{V})$ has *witness-extended emulation* if there is a PPT emulator $\mathcal{E}$ such that for every PPT prover $\mathcal{P}^*$ and every input x :

- Run $\mathcal{P}^*$ honestly with $\mathcal{V}$; let τ be the transcript and b the verifier’s accept/reject bit.
- Run $\mathcal{E}^{\mathcal{P}^*}(x)$; it outputs (τ', b', w) .

The distributions of (τ, b) and (τ', b') are computationally indistinguishable, and moreover whenever $b' = 1$, $(x, w) \in \mathcal{R}$ except with negligible probability.

WEE captures simultaneously two properties: “the extractor produces a witness whenever the protocol accepts” and “the extractor’s external behaviour resembles a normal protocol execution.” It is the appropriate notion for arguments embedded in larger protocols, such as the batched verification of many proofs at once (§5.6); plain knowledge soundness, however, suffices for this volume’s statements.

3.2 The algebraic group model

The IPA’s knowledge-soundness proof proceeds through the tree extractor of §5.5, with knowledge error $O(d/|\mathbb{F}|)$. The interactive tree extractor costs $3^{\log_2 d} = d^{\log_2 3}$ transcripts and runs in expected polynomial time (§3.3); the looseness arises for the non-interactive argument, where the naive Fiat–Shamir analysis of the $\log d$ -round protocol loses a factor $q^{O(\log d)}$ in the adversary’s random-oracle query count q — quasi-polynomial for polynomial q . The *algebraic group model* (AGM; Fuchsbauer, Kiltz, Loss; CRYPTO 2018) removes this loss by restricting the adversary’s group operations.

Definition 3.2 (Algebraic adversary). An *algebraic adversary* against a group $\mathbb{G}$ is a PPT algorithm that, whenever it outputs a group element $P \in \mathbb{G}$, additionally outputs an explicit linear combination $P = \sum_i [a_i]Q_i$ expressing P as a combination of previously-seen group elements Q_i .

The AGM sits between the standard model (no restriction on the adversary) and the generic group model (group operations available only through an oracle). Most natural adversaries are algebraic — they compute group elements by known scalar multiplications and additions — so the heuristic is regarded as conservative. In the AGM the forking loss disappears: an algebraic prover outputs every group element together with an explicit linear combination of the elements it has seen, so the extractor reads candidate witnesses directly off these representations instead of reassembling them from forked transcripts. Ghoshal and Tessaro (CRYPTO 2021) carry out this analysis for Bulletproofs-style inner-product arguments, proving tight state-restoration soundness — and hence tight Fiat–Shamir security — in the AGM. The accumulation analysis of BCMS20 (Bünz, Chiesa, Mishra, Spooner; TCC 2020), by contrast, does not invoke the AGM: it proves security in the random oracle model under the DL assumption (Theorem 5.7).

3.3 Tree-extractor analysis

To render the tree extractor concrete, consider the key step at each round of the IPA halving (see §5.5):

Lemma 3.3 (Single-round algebraic recovery). *Consider one node of a recursive IPA tree extractor. Suppose extraction below each child has already exposed the child’s folded scalar data, and the group messages have the algebraic representations required by the extraction model. In the symmetric normalisation of Construction 5.3, three branches whose nonzero challenges have pairwise distinct squares give an invertible system with rows $(u_j^{-2}, 1, u_j^2)$. In the deployed normalisation of Remark 5.4, the corresponding rows are $(u_j^{-1}, 1, u_j)$ and merely require pairwise distinct nonzero challenges. Solving the system recovers the parent representation and its cross-term consistency.*

Sketch. After the child data have been extracted, the three accepting relations are linear equations in the three parent coefficient terms. Multiplying a deployed row by u_j gives $(1, u_j, u_j^2)$, an ordinary Vandermonde row; its determinant is nonzero for distinct u_j . Multiplying a symmetric-normalisation row by u_j^2 gives $(1, u_j^2, u_j^4)$, so that version requires distinct squares. The raw group equations alone do not reveal discrete logarithms or witness halves; the conclusion uses the recursively extracted child data and the stated algebraic-representation or special-soundness hypotheses. $\square$

The full extraction requires three accepting transcripts at each of the $k = \log_2 d$ rounds, giving a tree of $3^k = d^{\log_2 3} \approx d^{1.585}$ leaves—polynomial in d , though more than the $O(d)$ a binary tree would cost. In the AGM the extractor can read algebraic representations directly. A standard-model or Fiat–Shamir extractor additionally needs the appropriate rewinding or state-restoration theorem; its cost cannot be obtained by simply multiplying every tree node by an independent $1/\epsilon$ factor.

3.4 Concrete versus asymptotic security

The literature on knowledge soundness contains numerous tightness gaps. The more important ones include:

- Bulletproofs’ naive Fiat–Shamir knowledge-soundness analysis loses a factor $q^{O(\log d)}$, exponential in the round count $\log d$ with base the query count q ; the interactive standard-model extraction itself is polynomial ($d^{\log_2 3}$ transcripts, expected polynomial time).
- Security analyses for Halo 2’s deployed parameters rely on the AGM and/or the random-oracle model.

This reflects the practical reality of deployed SNARKs: tight reductions in idealised models, loose reductions in standard models, and deployment selecting the former.

4 Recursive proof composition and accumulation schemes

We now turn to recursive proof composition: proving, inside one proof, that another proof verifies. This is what makes transparent Halo-style recursion efficient. Orchard’s mainnet deployment does not recurse. Its verifier can batch independent proof equations, but that randomized MSM batching is distinct from the recursive accumulation scheme developed here.

4.1 Recursion and its two classical obstacles

Consider a computation iterated many times, $z_n = f^{(n)}(z_0) := f(f(\dots f(z_0)\dots))$ — Valiant’s *incrementally-verifiable computation* (TCC 2008) — or, more generally, computations that branch and merge, each consuming the proofs of its predecessors — *proof-carrying data* (Chiesa, Tromer; ICS 2010). The natural construction is *recursive SNARK composition*: at step $i + 1$ the prover proves “ $z_{i+1} = f(z_i)$, and there exists a proof π_i that the verifier circuit for step i accepts.” One proof then attests to the entire history, and the per-step proving cost is independent of the history’s length.

For this to work, the SNARK’s verifier must itself be efficiently expressible as a circuit, and here the classical instantiations meet two obstacles:

1. Classical pairing-based systems such as Groth16 and KZG-backed PLONK have constant-size proofs and succinct verifiers, but their verifiers evaluate pairings—costly to express inside a circuit because they require extension-field arithmetic—and these particular systems use trusted setup.
2. Transparent SNARKs built on the IPA need no pairing and no trusted setup, but their verifiers run in linear time — the $O(d)$ MSM of §5.5.2. A linear-time check dominates the verifier circuit and negates the per-step efficiency.

Halo (Bowe–Grigg–Hopwood 2019) defeats obstacle 2 directly — deferring the linear-time part of verification instead of recomputing it at every step — and sidesteps obstacle 1 by using the IPA, which involves no pairing. BCMS20 formalised the deferral as an *accumulation scheme*.

4.2 Accumulation schemes

Definition 4.1 (Accumulation scheme, informal). For a predicate Φ (for example, “this IPA opening is correct”), an *accumulation scheme* consists of:

- An *accumulator*, a compact data structure representing a batch of claims about Φ .
- A *folding* algorithm: given an old accumulator and a new claim about Φ , it produces a new accumulator that represents both — the new accumulator is valid only if the old one was valid and the new claim holds. Folding is fast: $O(\log d)$ in the IPA instantiation below.

- A *decider*, which is expensive (linear-time) but which the scheme invokes only once at the very end to validate the accumulator. Under the scheme’s computational assumptions, an accepting decider implies that every folded claim is true, except with the stated soundness error.

For the IPA-PCS such a scheme exists, and it resolves the recursion problem in the IPA setting: the in-circuit verifier checks only the $O(\log d)$ “succinct” part of an IPA proof and folds the $O(d)$ “deferred” MSM check into a running accumulator. The decider pays the deferred work once, at the end of the entire chain. We state the scheme precisely as Theorem 5.7, once the IPA construction of §5.5 is in hand.

4.3 A cycle of curves for efficient native recursion

The accumulation scheme’s in-circuit folding involves arithmetic on group elements of the underlying curve. For this to be efficient inside the circuit, the group’s base field must equal the circuit’s native field. Since the circuit’s native field is the scalar field of the curve on which the prover commits the SNARK — which in turn is generally different from the curve’s own base field — a difficulty arises: an E_p -IPA proof’s group elements have coordinates in $\mathbb{F}_p$, so verifying it natively would require a circuit whose native field is $\mathbb{F}_p$. But a SNARK’s verifier circuit naturally operates over the curve’s *scalar* field $\mathbb{F}_q$ (where $q = |E_p|$): the Fiat–Shamir challenges and folding scalars are $\mathbb{F}_q$ elements. Curves with $p = q$ do exist — the *anomalous* curves, with trace of Frobenius 1 — but on them the discrete logarithm is computable in polynomial time (Smart; Semaev; Satoh–Araki, 1998–99), so no *secure* curve has $\mathbb{F}_p = \mathbb{F}_q$, and no one curve E_p can host both the proof’s coordinates and its own verifier circuit natively.

An efficient native resolution is a 2-cycle of elliptic curves, where $|E_p| = q$ and $|E_q| = p$. Then:

- A circuit committed on E_q verifies a proof generated over E_p ; that circuit’s native field is the scalar field of E_q , namely $\mathbb{F}_p$ (since $|E_q| = p$) — which coincides with the coordinate field of E_p , enabling native manipulation of E_p ’s group elements.
- A circuit over $\mathbb{F}_q$ verifies the next proof, generated over E_q .
- Proofs alternate between the two curves.

This is not a logical necessity: a circuit can emulate non-native field arithmetic, at substantial cost. It is the efficient native design used by the Pasta cycle of the *Math Guide*, where the two curves are Pallas (over $\mathbb{F}_{p_{\text{Pallas}}}$, order p_{Vesta}) and Vesta (over $\mathbb{F}_{p_{\text{Vesta}}}$, order p_{Pallas}).

5 The Halo 2 proof system

We now assemble the abstractions of the *Crypto Guide*–§4 into the concrete proof system Halo 2: a transparent PLONKish argument whose polynomial commitment uses one curve at a time. Orchard proves over the Pallas base field and commits on Vesta; the full Pasta cycle is needed only by the optional native-recursion construction above. This section develops Halo 2 to the level of detail required to state *what an Orchard Action proof actually is*.

5.1 PLONKish arithmetisation

Definition 5.1 (PLONKish constraint system). Fix a prime field $\mathbb{F}$ and an integer k defining a multiplicative subgroup $H := \{\omega^0, \dots, \omega^{n-1}\} \subset \mathbb{F}^*$ of size $n := 2^k$, where ω is a primitive n -th root of unity. A *PLONKish circuit* consists of:

- Columns partitioned into *fixed* (preprocessed), *advice* (prover-witness), and *instance* (public-input).

- A maximum constraint degree d .
- A finite sequence of *custom gates* $g_t(\vec{x}_0, \vec{x}_1, \dots) = 0$ — multivariate polynomials of degree $\leq d$ in column values at the current row and at offsets $\omega^r X$ for fixed small r .
- A subset $S \subseteq H \times [\text{\#cols}]$ of *equality-constraint cells* controlled by a permutation σ .
- A set of *lookup arguments*: input tuples $(A_0, \dots, A_{m-1})$ and table tuples $(S_0, \dots, S_{m-1})$, with the assertion that for every row $i \in [0, n)$, $(A_0(\omega^i), \dots, A_{m-1}(\omega^i))$ matches some row of the table.

Vanilla PLONK (Gabizon–Williamson–Ciobotaru, 2019) employs a fixed five-selector universal gate

$$q_M(X)a(X)b(X) + q_L(X)a(X) + q_R(X)b(X) + q_O(X)c(X) + q_C(X) \equiv 0 \pmod{Z_H(X)},$$

where $Z_H(X) := X^n - 1$ is the *vanishing polynomial* of the domain H (it is zero exactly on H , so “ $\equiv 0 \pmod{Z_H}$ ” means “zero at every row”). “PLONKish” generalises this by permitting designer-chosen gates of higher arity and degree, each gated by a selector $q_i(X)$ vanishing on rows where the gate is inactive. The Orchard Action circuit uses ten advice columns plus a small fixed-column lookup table for Sinsemilla; its custom gates encode the algebraic constraints of the Action statement together with the complete- and incomplete-addition arithmetic for in-circuit elliptic-curve operations (the *Math Guide* names the distinction).

Polynomial representation. The Lagrange basis $\ell_j(X)$ for H , where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$, interpolates each column into a polynomial of degree $< n$ over H . The prover commits to each polynomial via the IPA-PCS (§5.5). All subsequent arguments are *polynomial identities* on these committed polynomials, verifiable by querying them at random points.

5.2 From statement to circuit: arithmetisation in practice

Definition 5.1 states what a PLONKish circuit *is*; it does not say how a statement *becomes* one. The deployed code is organised into *chips*, reusable circuit components each packaging a set of columns, the gates and lookups declared on them, and the synthesis code that fills their cells. This subsection traces that passage: a real custom gate (the ECC chip’s incomplete point addition), two real lookup declarations (the ten-bit range check and the Sinsemilla generator table), and the synthesis machinery that turns witness data into the column vectors on which every argument of this section operates. The source versions audited here are `halo2_proofs 0.3.2`, `halo2_gadgets 0.5.0`, and `orchard 0.15.3`, with file paths relative to each crate’s `src/`. This audited Orchard release includes both the post-NU6.2 circuit correction and the post-NU6.3 circuit version.

Shape first, data second. A circuit, to `halo2_proofs`, is a type implementing the `Circuit` trait, whose two principal methods split the construction into phases (`halo2_proofs::plonk/circuit.rs`). The static method `configure` receives a mutable `ConstraintSystem` and declares the *shape*: it mints fixed, advice, and instance columns, allocates selectors, registers gates and lookups, and marks columns as equality-enabled. No witness exists at this point; `configure` is not even handed an instance of the circuit type. The method `synthesize` runs later, with the data, and fills cells. The split is the proof system’s division of labour: everything `configure` declares is public and determines the verifying key — the verifier’s entire view of the circuit — while everything `synthesize` writes into advice cells is the prover’s witness. (Key generation also runs `synthesize` once, against a witness-free backend that records fixed-cell values, selector activations, and copy constraints while ignoring advice; the prover runs the same code against a backend that records only advice. One synthesis procedure, two observers: `Assembly` in `plonk/keygen.rs` and `WitnessCollection` in `plonk/prover.rs`.)

Gates are queried identities. A call `meta.create_gate(name, closure)` defines one custom gate. Inside the closure, each `query_advice(column, Rotation(r))` — and its fixed and instance analogues — yields a symbolic handle to “the value of this column r rows below the gate’s own row”: the offsets $\omega^r X$ of Definition 5.1, realised at evaluation time by querying the column polynomial at $\omega^r x$ (`halo2_proofs, poly/domain.rs, rotate_omega`). The closure combines the handles by field arithmetic into an expression tree (Expression: constants, selector nodes, queried cells, negations, sums, products, scalings), one tree per constraint, and the helper `Constraints::with_selector` multiplies every constraint c by the gate’s selector expression, storing $q \cdot c$ (`plonk/circuit.rs`). This helper is the mechanical origin of the “selector times polynomial” shape of every deployed gate but one — the generalisation, gate by gate, of the fixed vanilla-PLONK equation of §5.1. (The exception, the ECC chip’s witness-point gate, multiplies by hand: its $(q \cdot x) \cdot (y^2 - x^3 - 5)$ and the helper’s $q \cdot (x \cdot (y^2 - x^3 - 5))$ are equal polynomials but different expression trees, and the pinned verifying key freezes the tree; `halo2_gadgets, ecc/chip/witness_point.rs`.) A selector is virtual at this stage: a named boolean column, off everywhere by default, enabled row by row during synthesis. At key generation, `compress_selectors` packs the selectors into genuine fixed columns — several to a column wherever the circuit’s degree bound leaves headroom — and rewrites every selector node into a fixed-column expression (`plonk/circuit.rs, plonk/keygen.rs`). The deployed Action circuit declares 56 selectors, whose compressed columns are among the 29 fixed columns pinned in its verifying key; the pinned count is the post-compression one (`orchard 0.15.3, src/circuit_data/circuit_description_fixed` and `circuit_description_post_nu6_3`).

Regions, the layouter, and wiring. Data enters at synthesis through a Layouter. A chip requests a *region* — a small rectangular window of cells addressed by relative offsets — fills it cell by cell, and leaves the choice of the region’s absolute starting row to the *floor planner* (`halo2_proofs, circuit.rs`). Values flow between regions not by position but by *copy constraints*: the call `enable_equality(column)` enrolls a column in the single global permutation of §5.3, and `constrain_equal` — or the assign-and-constrain combination `copy_advice` — merges the two cells’ cycles in that permutation (`plonk/permutation/keygen.rs`; the halo2 book’s *Permutation argument* chapter documents the cycle-merging). Public inputs bind the same way, an advice cell copy-constrained to an instance cell (§5.8). The deployed circuit uses the measuring V1 floor planner, which records every region’s shape in a witness-free pass, sorts the regions by advice area, and first-fits the largest ones first (`circuit/floor_planner/v1.rs`; the sort and first-fit in `v1/strategy.rs`); the `orchard` crate additionally pins the planner’s sort to a vendored Rust 1.56.1 `pdqsort` (feature `floor-planner-v1-legacy-pdqsort`) so that the layout — and with it the consensus verifying key — is identical across platforms and toolchains.

A deployed gate: incomplete point addition. The ECC chip’s incomplete-addition gate (`halo2_gadgets, ecc/chip/add_incomplete.rs`) computes $P + Q = R$ on Pallas under the promises $P, Q \neq \mathcal{O}$ and $x_p \neq x_q$. Its configuration is one selector $q_{\text{add-incomplete}}$ and four equality-enabled advice columns x_p, y_p, x_q, y_q ; the gate queries x_p, y_p, x_q, y_q at the current row and x_r, y_r at the next row of the *same* two columns x_{qr}, y_{qr} (hence their names: Q on the selector row, R below it). The two constraints, named x_r and y_r in the source, are

$$q_{\text{add-incomplete}} \cdot ((x_r + x_q + x_p)(x_p - x_q)^2 - (y_p - y_q)^2) = 0, \quad (1)$$

$$q_{\text{add-incomplete}} \cdot ((y_r + y_q)(x_p - x_q) - (y_p - y_q)(x_q - x_r)) = 0. \quad (2)$$

These are the affine chord formulas (the *Math Guide*) with the slope eliminated: for $x_p \neq x_q$ they are equivalent to $x_r + x_q + x_p = \lambda^2$ and $y_r + y_q = \lambda(x_q - x_r)$ with $\lambda := (y_p - y_q)/(x_p - x_q)$, multiplied through by $(x_p - x_q)^2$ and $(x_p - x_q)$ respectively to clear the denominators (the halo2 book’s *Addition* chapter records the same clearing). The raw degrees are three and two; with the selector factor, four and three — well inside the deployed circuit’s constraint-degree bound of nine (below). The *complete*-addition companion gate, which branches over the identity, doubling, and inverse cases that this gate

excludes, needs twelve constraint polynomials of degree up to six and five auxiliary witness cells per invocation (`ecc/chip/add.rs`); incompleteness is a bargain wherever its promises can be kept.

Synthesis for one invocation (`assign_region`, same file) makes the witness path concrete. The chip enables the selector at the region’s row i ; copies the four input coordinates into that row with `copy_advice`, which both assigns each cell and copy-constrains it back to its source; computes, *out of circuit*,

$$\lambda = \frac{y_q - y_p}{x_q - x_p}, \quad x_r = \lambda^2 - x_p - x_q, \quad y_r = \lambda(x_p - x_r) - y_p;$$

and assigns x_r, y_r into row $i + 1$ (Table 1). The slope λ is never written to any cell: the prover uses it to compute the answer, and the committed identities (1)–(2) verify the answer without it. The division is not even performed here — coordinates are carried as deferred fractions (`halo2_proofs`, `plonk/assigned.rs`), and every denominator falls to a single batch inversion when synthesis ends (`plonk/prover.rs`, `batch_invert_assigned`). The output cells are returned as a new non-identity point with no further on-curve check — sound because, once the call site guarantees distinct x -coordinates on non-identity inputs, the two identities force $R = P + Q$, a non-identity curve point.

row	x_p	y_p	x_qr	y_qr	q _{add-incomplete}
i	x_p	y_p	x_q	y_q	1
$i + 1$	–	–	x_r	y_r	0

Table 1: The incomplete-addition region: two rows by four advice columns (`halo2_gadgets`, `ecc/chip/add_incomplete.rs`). The four input cells x_p, y_p, x_q, y_q on the selector row i are copy-constrained to the cells they came from; the result cells x_r, y_r occupy the `x_qr`, `y_qr` columns of row $i + 1$, where the gate’s `Rotation::next()` queries find them. Dashes mark cells the gate does not use.

Incompleteness, and where it is discharged. Reading the two polynomials at the excluded inputs shows exactly what “incomplete” costs. If $Q = -P \neq \mathcal{O}$ (so $x_p = x_q$ and $y_p = -y_q \neq 0$), constraint (1) collapses to $-(2y_p)^2 \neq 0$: no assignment satisfies the gate — a completeness failure, never a soundness one. If $Q = P$, both polynomials vanish identically and (x_r, y_r) is *unconstrained*: a reachable doubling would let the prover claim any result. If an input were the identity in the gadget’s $(0, 0)$ affine encoding, the constraints would accept a garbage output. The deployed circuit discharges the three exclusions in three different places. Non-identity travels with the type: the gate’s inputs are `NonIdentityEccPoint` values. A fresh point entering the chip is witnessed under the gate

$$q_{\text{point-non-id}} \cdot (y^2 - x^3 - 5) = 0 \quad (\text{halo2_gadgets, ecc/chip/witness_point.rs}),$$

which $(0, 0)$ cannot satisfy because 5 is not a square and -5 is not a cube in $\mathbb{F}_{p_{\text{Pallas}}}$ (the `halo2` book’s *Addition* chapter) — but no fresh witness ever reaches incomplete addition. In the deployed circuit the incomplete-addition gate fires only inside fixed-base scalar multiplication (`ecc/chip/mul_fixed.rs`), which sums precomputed windowed multiples $[(k + 2) \cdot 8^w]B$ of a fixed base B . There the addends are assembled by the type’s unchecked constructor (`from_coordinates_unchecked`) and pinned instead to the precomputed window table by `mul_fixed`’s own coordinate gates (`coords_check`, same file): the x -coordinate must equal a degree-seven polynomial in the window value whose fixed coefficients interpolate the window’s eight table x -coordinates, and the y -coordinate must satisfy $y + z_w = u^2$ against a per-window fixed z_w and a witnessed u , alongside an on-curve check — together pinning the point to the table entry selected by the window digit, and no entry of the table is the identity. The running accumulator is a sum of such entries under this very gate; the type carries the non-identity guarantee from either construction site to the gate, which re-checks nothing. The distinctness $x_p \neq x_q$ is enforced by no constraint at all; it too is structural: the window tables store $[(k + 2) \cdot 8^w]B$ rather than the naive $[(k + 1) \cdot 8^w]B$ precisely

because the naive offset admits a window collision that produces the fatal doubling (the halo2 book’s *Fixed-base scalar multiplication* chapter). Prover-side, `assign_region` rejects a *known* bad witness with an error — a developer diagnostic, not a constraint. (The circuit’s other incomplete additions — the variable-base ladder’s fused double-and-add rounds and Sinsemilla’s two per chunk — are separate gates built on the same discipline; `ecc/chip/mul/incomplete.rs`, `sinsemilla/chip.rs`.)

Non-algebraic relations become lookups. A gate can assert only polynomial identities. “This value has ten bits” and “this point is the j -th Sinsemilla generator” are not identities but set memberships, and the circuit declares them as lookups: a call `meta.lookup(closure)` returns pairs of an input *expression* and a table column, and pushes one lookup argument onto the constraint system (`halo2_proofs`, `plonk/circuit.rs`). Each declaration is one instance of the argument of §5.4, the input expressions playing $A_0, \dots, A_{m-1}$ and the table columns $S_0, \dots, S_{m-1}$. The deployed circuit carries exactly three (`orchard 0.15.3`, `src/circuit_data/circuit_description_post_nu6_3`): one shared range-check declaration, and the Sinsemilla generator declaration twice — once per Sinsemilla chip instance, each on its own half of the ten advice columns.

All three consume the same table. Loaded once at synthesis, it holds, for $0 \leq j < 2^{10}$,

$$\text{row } j : \quad (j, X(S[j]), Y(S[j])), \quad S[j] := \text{GroupHash}(\text{"z.cash:SinsemillaS"}, \text{LE}_{32}(j)),$$

in three table columns $(t_{\text{idx}}, t_x, t_y)$; the generators ship as precomputed constants that the crate’s own tests re-derive from the hash (`sinsemilla crate`, `src/sinsemilla_s.rs`; `halo2_gadgets`, `utilities/lookup_range_check.rs` for the loading). After the assignment, the layouter fills every remaining usable row of each table column with that column’s row-0 value (`halo2_proofs`, `circuit/table_layouter.rs`); a defaulted input row therefore still matches a genuine table row, and the type `TableColumn` hides its underlying fixed column exactly so that no chip can bypass this default-filling.

The range check is one declaration serving two modes (`halo2_gadgets`, `utilities/lookup_range_check.rs`):

$$q_{\text{lookup}} \cdot (q_{\text{running}} \cdot (z_i - 2^{10} z_{i+1}) + (1 - q_{\text{running}}) \cdot z_i) \mapsto t_{\text{idx}}, \quad (3)$$

with z_i and z_{i+1} the running-sum advice column at the current and next rotations. With $q_{\text{running}} = 1$ the input is a running-sum step: the prover witnesses $z_{i+1} = (z_i - a_i) \cdot 2^{-10}$ on successive rows, the expression recovers the limb $a_i = z_i - 2^{10} z_{i+1}$, and membership in $t_{\text{idx}} = \{0, \dots, 2^{10} - 1\}$ asserts that each limb of the decomposition has ten bits; a *strict* decomposition additionally copy-constrains the final z to the constant 0, range-constraining the decomposed element to $10w$ bits for w limbs. With $q_{\text{running}} = 0$ the cell itself is looked up (checks of fewer than ten bits add a small bit-shift gate in the same file). Every range check in the Action circuit funnels through this one declaration, on a single advice column.

The Sinsemilla declaration is the three-column instance (`halo2_gadgets`, `sinsemilla/chip/generator_table.rs`):

$$(q_{S1} \cdot m, q_{S1} \cdot x_p + (1 - q_{S1}) \cdot X(S[0]), q_{S1} \cdot y_p + (1 - q_{S1}) \cdot Y(S[0])) \mapsto (t_{\text{idx}}, t_x, t_y), \quad (4)$$

where $m = z_i - 2^{10} \cdot q_{\text{run}} \cdot z_{i+1}$ recovers the ten-bit message word from the hash’s own running-sum decomposition (with $q_{\text{run}} := q_{S2} - q_{S2}(q_{S2} - 1)$, an expression in a fixed column $q_{S2} \in \{0, 1, 2\}$ that evaluates to 1 on the interior rows of a message piece and to 0 on final rows, where the trailing running sum is implicitly zero), and where y_p is not a witnessed cell at all but the derived expression $Y_A \cdot 2^{-1} - \lambda_1(x_A - x_p)$ over the chip’s double-and-add columns (`ecc/chip/mul/incomplete.rs` supplies $Y_A := (\lambda_1 + \lambda_2)(x_A - x_R)$ and $x_R := \lambda_1^2 - x_A - x_p$). Lookup inputs, in other words, may be arbitrary expressions over queried cells, not merely cells; the chip proves that the generator it is adding is $S[m]$ without ever spending an advice cell on $y_{S[m]}$. The $(1 - q_{S1})$ branches default every row on which the chip is inactive to table row 0, so the membership assertion of §5.4 holds on all rows.

One API restriction is visible here: the selectors gating lookup inputs (q_{lookup} , q_{running} , q_{S1}) must be declared as `complex_selectors` — simple selectors are reserved for gates, where the key-generation compression is licensed to rewrite them (`halo2_proofs, plonk/circuit.rs`).

The handoff. When `synthesize` returns, the construction has produced nothing exotic: every column — advice, fixed, instance — is a vector of $n = 2^k$ field elements; in the advice columns the deferred fractions are resolved by one batch inversion and the reserved trailing rows filled with blinding randomness (§5.9, Remark 5.2). The prover commits to each advice column directly in the Lagrange basis (`halo2_proofs, poly/commitment.rs, commit_lagrange` — the same group element as committing the interpolated coefficients, so the commitment even precedes interpolation) and then interpolates each column over H by an inverse FFT (`poly/domain.rs, lagrange_to_coeff`): these are precisely the column polynomials of §5.1. Each gate’s stored expression tree is then evaluated with the committed column polynomials substituted for the queried cells — a query at rotation r reading the polynomial at $\omega^r X$ — and the results are folded with powers of the challenge y and divided by $Z_H(X) = X^n - 1$ (`plonk/prover.rs; plonk/vanishing/prover.rs; divide_by_vanishing_poly` in `poly/domain.rs`). The gate identities declared in `configure` have become the quotient $h(X)$ of Phase 4 (§5.7). This is the point where the worked example of §6.3 resumes: it interpolates a four-row trace and assembles exactly this $G(X)$ whose divisibility by Z_H the rest of the protocol certifies — and where Orchard’s deployed gadget stack bottoms out: each gadget is a bundle of configure-time gates and lookups plus `synthesize`-time regions, and after synthesis nothing remains of the stack but rows in these same columns. The scale-up from the toy is numerical, not conceptual. The deployed Action circuit has $k = 11$ ($n = 2048$ rows), the ten advice columns of §5.1, one instance column, and twenty-nine fixed columns, with a constraint-degree bound of nine across its gates and permutation and lookup expressions — so the quotient splits into $9 - 1 = 8$ chunks, and the prover’s FFTs run over an extended domain of size $2^{14} = 8 \cdot 2^{11}$ (`orchard 0.15.3, src/circuit_data/circuit_description_post_nu6_3; halo2_proofs, poly/domain.rs`). The next two subsections supply the arguments this construction leans on: the permutation argument behind every copy constraint, and the lookup argument behind declarations (3) and (4).

5.3 The permutation (equality) argument

A PLONK-style permutation argument, which Halo 2 generalises to multiple columns, enforces equality constraints between distinct cells. Each cell (i, j) (column i , row j) receives a unique label $\delta_i \cdot \omega^j$ with δ_i chosen so all products are distinct as (i, j) ranges over the trace. Preprocessing fixes σ -permuted polynomials $s_i(X)$ with $s_i(\omega^{j'}) = \delta_{i'} \cdot \omega^{j'}$ when $\sigma(i, j) = (i', j')$; the verifying key contains commitments to them.

Given Fiat–Shamir challenges $\beta, \gamma \in \mathbb{F}$, the prover commits to a running-product polynomial $Z_{\text{perm}}(X)$ asserting, where $v_i(X)$ denotes the Lagrange interpolation (§5.1) of the i -th column participating in the permutation,

$$\prod_{i,j} (v_i(\omega^j) + \beta \delta_i \omega^j + \gamma) = \prod_{i,j} (v_i(\omega^j) + \beta s_i(\omega^j) + \gamma).$$

The assignment satisfies every equality constraint (cells related by σ carry equal values) precisely when the denominator factors are a permutation of the numerator factors, in which case the product equals 1 for every β, γ ; conversely, if some constraint is violated, the two products at the sampled β, γ are equal with probability at most $O(N/|\mathbb{F}|)$ by Schwartz–Zippel, where N is the number of cells participating in the permutation. The verifier checks this by opening Z_{perm} at x and ωx , verifying the boundary $Z_{\text{perm}}(\omega^0) = 1$ and a cross-multiplied row-by-row update, which remains defined when a factor is zero. Excluding the final IPA MSM, this argument contributes $O(m + \log n)$ verifier work, where m is the number of participating columns; the complete deployed verifier still has the $O(n)$ commitment check of §5.5.2.

5.4 The lookup argument

Halo 2’s lookup argument is structurally simpler than plookup (Gabizon–Williamson 2020) and supports arbitrary (possibly non-fixed) tables.

Objective. Establish that every entry of column $A(X)$ on the active domain appears as some entry of column $S(X)$.

Construction. The prover supplies polynomials $A'(X), S'(X)$ where (i) A' is a permutation of A with like values grouped into contiguous runs, and (ii) S' is a permutation of S in which the first row of each run of like values in A' has the matching value in S' . The prover commits to A' and S' ; after receiving challenges β, γ , it commits to a running-product column $Z_{\text{lookup}}(X)$ and proves:

$$\begin{aligned} \ell_0(X)(1 - Z_{\text{lookup}}(X)) &= 0, \\ Z_{\text{lookup}}(\omega X)(A'(X) + \beta)(S'(X) + \gamma) - Z_{\text{lookup}}(X)(A(X) + \beta)(S(X) + \gamma) &= 0, \\ \ell_0(X)(A'(X) - S'(X)) &= 0, \\ (A'(X) - S'(X))(A'(X) - A'(\omega^{-1}X)) &= 0. \end{aligned}$$

The first two constraints, with the boundary conditions of Remark 5.2, certify with the usual Schwartz–Zippel error that A' and S' are permutations of A and S respectively; the third aligns row 0 ($A'_0 = S'_0$), where the wrap-around reference $A'(\omega^{-1}X)$ deprives the fourth constraint of force. The fourth constrains every subsequent row either to start a new run aligned with S' ($A'_i = S'_i$) or to continue the previous one ($A'_i = A'_{i-1}$). Inductively, every value of A equals some value of S' , hence of S .

A random-linear combination with a challenge θ reduces multi-column lookups to single-column: $A_c(X) := \sum_j \theta^{m-1-j} A_j(X)$. Variable tables (where S is itself an advice column) and range checks (where S enumerates the allowed range) are special cases.

Remark 5.2 (Active rows and blinding). The lookup constraints above, like the permutation constraints of §5.3, hold on the *active* rows only: the deployed system reserves the trailing rows of every column for random blinding values (§5.9). Each product-update constraint is therefore multiplied by the factor $(1 - (\ell_{\text{last}}(X) + \ell_{\text{blind}}(X)))$, where ℓ_{blind} and ℓ_{last} are Lagrange-basis selectors of the blinding rows and of the boundary row immediately before them — computed directly from the domain by both parties, not committed circuit columns — and boundary constraints such as $\ell_{\text{last}}(X)(Z(X)^2 - Z(X)) = 0$ pin the running product to $\{0, 1\}$ at the boundary row. The random rows consequently neither violate the arguments nor weaken them.

5.5 The inner-product polynomial commitment

The Halo 2 polynomial commitment, instantiating the abstract PCS of the *Crypto Guide*, is the inner-product argument of Bootle–Cerulli–Chaidos–Groth–Petit (EUROCRYPT 2016) as refined in Bulletproofs (Bünz et al., 2017). We give the construction in detail. Write $d := 2^k$ for the degree bound (equivalently, the number of generators); the round count $k = \log_2 d$ reuses the letter k of Definition 5.1, harmlessly, because in the assembled protocol of §5.7 the committed polynomials have degree $< n$, so $d = n = 2^k$ there. The letter H likewise does double duty, inherited from two literatures: $H \subset \mathbb{F}$ is the PLONK evaluation domain (as in Z_H), while $H \in \mathbb{G}$ is the Pedersen blinding generator of the *Crypto Guide*; the ambient set ($\mathbb{F}$ versus $\mathbb{G}$) disambiguates every occurrence. Two further letters carry double duty: p denotes the group order (as in $\mathbb{F}_p$) and, with an argument or in $\deg p$, the committed polynomial $p(X)$; and the permutation polynomials $s_i(X) \in \mathbb{F}[X]$ of §5.3 are distinct from the IPA structured scalars $s_i \in \mathbb{F}_p$ of §5.5.2. Type and argument disambiguate every occurrence.

Construction 5.3 (IPA-based polynomial commitment). Let $\mathbb{G}$ be a cyclic group of prime order p in which DL is hard. Choose $d = 2^k$ and sample, via hash-to-curve, $\vec{G} = (G_1, \dots, G_d)$ and $H \in \mathbb{G}$ with no known non-trivial linear relation among them.

For $p(X) = \sum_{i=0}^{d-1} a_i X^i \in \mathbb{F}_p[X]$ and blinder $r \in \mathbb{F}_p$,

$$\text{Commit}(p; r) := \langle \vec{a}, \vec{G} \rangle + [r]H \in \mathbb{G}.$$

To open p at $x \in \mathbb{F}_p$ with claimed value v , the relation is

$$\mathcal{R}_{\text{IPA}} := \left\{ ((P, x, v); (\vec{a}, r)) : P = \langle \vec{a}, \vec{G} \rangle + [r]H, v = \langle \vec{a}, \vec{b}(x) \rangle, \deg p \leq d-1 \right\},$$

with $\vec{b}(x) := (1, x, x^2, \dots, x^{d-1})$.

5.5.1 The recursive halving protocol

The argument runs in $k = \log_2 d$ rounds. Initialise $\vec{a}^{(k)} := \vec{a}$, $\vec{G}^{(k)} := \vec{G}$, $\vec{b}^{(k)} := \vec{b}(x)$, and let $U \in \mathbb{G}$ be a challenge group element (in Fiat–Shamir, derived from the transcript). The prover absorbs $[v]U$ so that the relation becomes

$$P + [v]U = \langle \vec{a}, \vec{G} \rangle + [r]H + [\langle \vec{a}, \vec{b} \rangle]U.$$

In round j (descending from k to 1): split each vector into halves and let

$$L_j := \langle \vec{a}_{\text{lo}}^{(j)}, \vec{G}_{\text{hi}}^{(j)} \rangle + [\ell_j]H + [\langle \vec{a}_{\text{lo}}^{(j)}, \vec{b}_{\text{hi}}^{(j)} \rangle]U,$$

$$R_j := \langle \vec{a}_{\text{hi}}^{(j)}, \vec{G}_{\text{lo}}^{(j)} \rangle + [r_j]H + [\langle \vec{a}_{\text{hi}}^{(j)}, \vec{b}_{\text{lo}}^{(j)} \rangle]U.$$

The verifier returns a uniform $u_j \in \mathbb{F}_p^*$. Both parties fold:

$$\vec{a}^{(j-1)} := u_j \vec{a}_{\text{lo}}^{(j)} + u_j^{-1} \vec{a}_{\text{hi}}^{(j)},$$

$$\vec{G}^{(j-1)} := u_j^{-1} \vec{G}_{\text{lo}}^{(j)} + u_j \vec{G}_{\text{hi}}^{(j)},$$

$$\vec{b}^{(j-1)} := u_j^{-1} \vec{b}_{\text{lo}}^{(j)} + u_j \vec{b}_{\text{hi}}^{(j)}.$$

After k rounds each vector has length 1. The prover sends the final scalar $a := \vec{a}^{(0)}$ and an aggregated blinder r^* . The verifier accepts iff

$$P + [v]U + \sum_{j=1}^k ([u_j^2]L_j + [u_j^{-2}]R_j) \stackrel{?}{=} [a]G^{(0)} + [r^*]H + [a \cdot b^{(0)}]U, \quad (5)$$

where $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ and the challenges determine $\vec{s}$ as below.

5.5.2 The structured scalars and the polynomial g

Write the binary expansion $i = \sum_{\ell=0}^{k-1} b_\ell 2^\ell$ for $i \in [0, d)$. By induction on the folding,

$$G^{(0)} = \langle \vec{s}, \vec{G} \rangle, \quad b^{(0)} = \langle \vec{s}, \vec{b}(x) \rangle = g(x; u_1, \dots, u_k),$$

where

$$s_i := \prod_{\ell=0}^{k-1} u_{\ell+1}^{(-1)^{1-b_\ell}}, \quad g(X; u_1, \dots, u_k) := \prod_{\ell=1}^k (u_\ell^{-1} + u_\ell X^{2^{\ell-1}}).$$

The essential asymmetry is the following: $g(X)$ has degree $d-1$ and the verifier can evaluate it at x in $O(\log d)$ field operations; but computing $G^{(0)} = \text{Commit}(g; 0)$ requires an $O(d)$ -size multi-scalar multiplication (MSM). This asymmetry is the seed of the accumulation scheme (§5.6).

Remark 5.4 (The deployed IPA normalisation). The deployed opening (halo2_proofs 0.3.2, poly/commitment/prover.rs and verifier.rs) uses the rescaled Halo convention, not the Bulletproofs one above: the folding weights are $(1, u_j^{-1})$ on the coefficient vector and $(1, u_j)$ on $\vec{b}$ and $\vec{G}$, the verifier accumulates $[u_j^{-1}]L_j + [u_j]R_j$, and the deferred polynomial is $g(X) = \prod_{i=0}^{k-1} (1 + u_{k-1-i}X^{2^i})$. The opening also begins with an extra blinding commitment S and two challenges ξ, z , and ends with two scalars: the collapsed coefficient c and a synthetic blinder f . The two conventions are algebraically related by per-round rescaling and a formal relabelling of the challenge. This does *not* identify their challenge distributions: a uniform field element in the code is not the square of a uniform field element over all of $\mathbb{F}_p$. Consequently extraction and soundness conditions must be stated for the deployed coefficients $u_j^{-1}, 1, u_j$, as in Lemma 3.3, rather than transferred automatically from the square-weighted presentation. The code squeezes full-field challenges without rejection; a zero u_j makes the required inversion fail, and analogous denominator collisions occur elsewhere with negligible probability. Thus its implementation-level completeness is overwhelming rather than literally perfect.

5.5.3 Worked example: an IPA opening at $d = 4$

Take $d = 4$, so $k = 2$ rounds. Let $p(X) = a_0 + a_1X + a_2X^2 + a_3X^3$ with $\vec{a} = (a_0, a_1, a_2, a_3)$. The verifier holds $P = \langle \vec{a}, \vec{G} \rangle + [r]H$ (we suppress r to declutter; the H -part propagates identically) and requires $v = p(x)$ at some $x \in \mathbb{F}_p$.

Set $\vec{b}^{(2)} := (1, x, x^2, x^3)$ and $\vec{G}^{(2)} := \vec{G}$. After the prover absorbs $[v]U$, the relation is $P + [v]U = \langle \vec{a}, \vec{G} \rangle + \langle \vec{a}, \vec{b}^{(2)} \rangle U$.

Round 2. Split into halves of size 2. The prover sends

$$\begin{aligned} L_2 &= a_0G_3 + a_1G_4 + (a_0x^2 + a_1x^3)U, \\ R_2 &= a_2G_1 + a_3G_2 + (a_2 + a_3x)U. \end{aligned}$$

The verifier picks $u_2 \in \mathbb{F}_p^*$ and both parties fold to length-2 vectors as above.

Round 1. Split the length-2 vectors. The prover sends L_1, R_1 analogously; the verifier picks u_1 . After folding, all three vectors have length 1.

Final check. With $i = b_0 + 2b_1$ ranging over $\{0, 1, 2, 3\}$:

$$s_0 = u_1^{-1}u_2^{-1}, \quad s_1 = u_1u_2^{-1}, \quad s_2 = u_1^{-1}u_2, \quad s_3 = u_1u_2.$$

The verifier evaluates $g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2)$ in four field multiplications (u_1x, x^2, u_2x^2 , and the product of the two factors), and computes $G^{(0)} = \sum_{i=0}^3 [s_i]G_{i+1}$ as a length-4 MSM. The check (5) becomes

$$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 \stackrel{?}{=} [a]G^{(0)} + [a \cdot g(x; u_1, u_2)] U.$$

The folding rule for $\vec{G}$ ensures that $G^{(0)}$ is a specific linear combination of the original $\vec{G}$ with coefficients $\vec{s}$; that the s_i match $\prod_{\ell} (u_{\ell}^{-1} \text{ or } u_{\ell})$ according to the binary expansion of i is exactly the closed-form identity. The verifier can evaluate g in $O(\log d)$ field operations (about $3k-2$: k coefficient products, $k-1$ squarings of x , $k-1$ factor multiplications) but recomputing $G^{(0)}$ takes $O(d)$ MSM — the asymmetry the accumulator defers.

5.5.4 Knowledge soundness via rewinding

Theorem 5.5 (Knowledge soundness of IPA). *Under the DL assumption on $\mathbb{G}$, the interactive opening protocol of Construction 5.3 is knowledge-sound for $\mathcal{R}_{\text{IPA}}$ with knowledge error $O(d/|\mathbb{F}_p|)$, via an expected-polynomial-time tree extractor. The Fiat–Shamir-compiled argument inherits the bound only up to the naive $q^{O(\log d)}$ random-oracle query loss; the tight non-interactive bound needs the AGM (§3).*

Sketch of the tree extractor. The extractor recursively obtains the folded scalar data below each child, then rewinds the prover $\mathcal{P}^*$ to obtain three accepting branches that agree on (L_j, R_j) . Lemma 3.3 then inverts the resulting linear system to recover the parent representation. For Construction 5.3, the three nonzero challenges need pairwise distinct squares because the coefficients are $u_j^{-2}, 1, u_j^2$; for the deployed normalisation, pairwise distinct nonzero values suffice because the coefficients are $u_j^{-1}, 1, u_j$. Three raw group equalities alone would not reveal the scalar halves: recursive child extraction and the protocol’s special-soundness or algebraic-representation hypotheses are essential.

Extracting the full witness $\vec{a} \in \mathbb{F}_p^d$ requires three sibling transcripts at the deepest level, which yield one round-1 witness; three such triples, hung off pairwise distinct challenges at the next-shallower round, yield one round-2 witness; and so on, each shallower round tripling the requirement. The tree thus has $3^k = d^{\log_2 3}$ leaves — polynomial in d (§3) — yielding polynomial expected extraction time when $d = \text{poly}(\lambda)$. The theorem’s knowledge error of $O(d/|\mathbb{F}_p|)$ does not come from a tree-wide union bound. Each of the $k = \log_2 d$ rounds excludes $O(1)$ bad challenges (equal squares in the symmetric presentation, or equal values in the deployed one), contributing $O(\log d/|\mathbb{F}_p|) \leq O(d/|\mathbb{F}_p|)$; Schwartz–Zippel terms of degree d account for the rest. Collisions among the resampled challenges inside the extractor’s 3^k -leaf tree affect only its expected running time — the extractor rewinds and resamples — not the knowledge error; a naive union bound over the roughly 3^k sibling challenge draws would bound the tree-wide collision probability by $O(d^{\log_2 3}/|\mathbb{F}_p|)$, still negligible, but that is not the source of the theorem’s figure.

This is the recursive generalisation of special soundness for Schnorr (the *Crypto Guide*): a single transcript reveals nothing, but a few transcripts on the same first message permit inversion of a small linear system; in the IPA the system is 3×3 per round and the recursion stacks k rounds. $\square$

5.6 Accumulation and the Halo trick

The verifier’s check (5) runs in $O(\log d)$ scalar operations and $\log d$ point additions, *except* for the single evaluation $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ — an MSM of length d . For optional recursive composition, the accumulation idea (Bowe–Grigg–Hopwood 2019; formalised by BCMS20) is to *defer* this MSM and amortise it across many recursively composed proofs.

Definition 5.6 (Atomic accumulator for IPA). An IPA *accumulator instance* is a tuple $\text{Acc} := (G^{(0)}, u_1, \dots, u_k) \in \mathbb{G} \times \mathbb{F}_p^k$. It is *valid* iff $G^{(0)} = \langle \vec{s}(\vec{u}), \vec{G} \rangle = \text{Commit}(g(X; u_1, \dots, u_k); 0)$ — the deferred claim concerns the unblinded commitment, the blinder having been discharged by the $[r^*]H$ term of (5). Subsequent unary $\text{Commit}(\cdot)$ abbreviates $\text{Commit}(\cdot; 0)$.

Theorem 5.7 (IPA accumulation, informal BCMS20 interface). *BCMS20 constructs an accumulation scheme $(\text{AccVerifier}, \text{AccDecider})$ for IPA with:*

- *AccVerifier, on input a new IPA instance and an old accumulator, runs in $O(\log d)$ and outputs a new accumulator Acc' .*
- *AccDecider, run once at the end of a chain, performs an MSM of length d to check $G^{(0)} = \text{Commit}(g)$.*

- Soundness reduces to DL on $\mathbb{G}$ in the random-oracle model.

Operational interpretation. A Halo 2 verifier circuit, expressed over the scalar field of one Pasta curve and verifying a proof produced over the *other* curve, can:

1. Run the $O(\log d)$ “succinct” checks of the IPA verifier natively.
2. Witness the next-step claimed $G^{(0)'}$ and the next challenges $u'_1, \dots, u'_k$ as public inputs to the next proof, deferring the MSM check.
3. Fold the deferred MSM checks: with a random challenge ρ , the two claims $G^{(0)} = \text{Commit}(g)$ and $G^{(0)'} = \text{Commit}(g')$ reduce to the single check $G^{(0)} + [\rho] G^{(0)'} = \text{Commit}(g + \rho g')$, with both challenge tuples retained so that the combined length- d coefficient vector $\vec{s}(\vec{u}) + \rho \vec{s}(\vec{u}')$ is recomputable; a false claim survives the combination with probability at most $1/|\mathbb{F}_p|$. In the recursive instantiation the combined commitment is opened at a fresh random point inside the next proof’s multipoint argument — each g_i there is evaluable in $O(\log d)$ — so the deferred claim emerging from the next IPA opening is again a single tuple of the form of Definition 5.6; the tuple type is closed under accumulation only through this prover-assisted step, not under verifier-side linear combination.

A single final MSM at the end of the chain validates the entire history. The Pasta cycle (§4.3) renders each step native: a Vesta-IPA proof’s circuit, whose native field is the Vesta scalar field $\mathbb{F}_{p_{\text{Pallas}}}$, verifies the previous Pallas-IPA proof, whose group elements (the column and quotient commitments, the (L_j, R_j) pairs, the accumulator’s $G^{(0)}$) have coordinates in exactly that field; the next Pallas proof verifies the Vesta proof by the symmetric argument. This cross-curve alternation is what enables proof-carrying data without expensive non-native field simulation. The same coincidence is what lets Orchard’s $\mathbb{F}_{p_{\text{Pallas}}}$ -native Action circuit manipulate the Pallas points cm , rk , and cv^{net} directly (§5.8).

Orchard’s deployment does not recurse: the verifier of an Action proof pays the deferred MSM itself. The library’s batch verifier separately random-scales and adds the complete MSM equation produced for each proof, then evaluates one combined MSM. This is randomized batch verification; it neither emits an accumulator for a later proof nor invokes the BCMS recursive accumulation scheme. Both techniques use linear combinations, but their interfaces and soundness statements are different.

The deferred equation $G^{(0)} = \text{Commit}(g(X; u_1, \dots, u_k); 0)$ is a commitment-equality (equivalently, an MSM) claim for a publicly structured coefficient vector. It is not itself an evaluation opening. The accumulation protocol reduces and carries this claim using additional prover-assisted steps.

5.7 The complete Halo 2 protocol

We now assemble the pieces into the seven-phase protocol implemented in the halo2 code-base. Throughout, $\mathbb{F}$ denotes the constraint field (Orchard: $\mathbb{F}_{p_{\text{Pallas}}}$), $\mathbb{G}$ the IPA group (Orchard: $\text{Vesta}(\mathbb{F}_{p_{\text{Vesta}}})$), and H the multiplicative subgroup of $\mathbb{F}$ of order n .

Setup (preprocessed, one-time). The circuit designer specifies columns, gates, lookup tables, and the equality permutation σ . Selector compression materialises selectors in fixed columns, and assigned lookup tables are fixed-column data; they are not two additional disjoint classes of committed columns. Key generation commits to the resulting fixed-column polynomials and to the permutation polynomials. The transparent IPA parameters separately contain the domain data and generator bases. At the generic `halo2_proofs` API boundary, verification receives those parameters, a PLONK verifying key, the instance values, and the proof. Orchard’s higher-level `VerifyingKey` wrapper contains both the IPA parameters and the PLONK key, so its public interface can present those as one argument. All are reproducible from public algorithms and the circuit; no secret setup trapdoor exists.

Phase 1 — advice commitments. Both parties first absorb a digest of vk and the (unblinded) instance commitments into the transcript (§5.8), so every subsequent challenge depends on the circuit and the public input. The prover then constructs the advice column values $\vec{w}_i \in \mathbb{F}^n$ satisfying all gates and equality constraints; fills the reserved trailing rows with uniformly random field elements (the blinding rows of §5.9); interpolates each $\vec{w}_i$ to a polynomial $w_i(X)$ of degree $< n$ on H ; commits via IPA-PCS with a fresh blinder; sends the commitments.

Phase 2 — lookup compression: challenge θ . The verifier (via Fiat–Shamir, here and throughout) returns $\theta \in \mathbb{F}$. For each lookup the prover compresses multi-column inputs and tables with powers of θ , computes the permuted columns A', S' (§5.4), and commits to them.

Phase 3 — running products: challenges β, γ . The verifier returns $\beta, \gamma \in \mathbb{F}$. The prover commits to the permutation running product Z_{perm} (§5.3) and to one lookup running product Z_{lookup} per lookup (§5.4). The order matters for soundness: the permuted columns A', S' must be fixed *before* the challenges β, γ that randomise the products are drawn.

Phase 4 — quotient challenge y . Before y is drawn, the prover commits to the random polynomial used to blind the quotient opening. The verifier then returns a fresh $y \in \mathbb{F}$. The prover assembles the *gate equation*: a single polynomial $C(X)$ that is a y -weighted linear combination of (i) each custom gate, (ii) permutation constraints, (iii) lookup constraints. By construction $C(X)$ vanishes on H when all gates pass, so the quotient $h(X) := C(X)/Z_H(X) = C(X)/(X^n - 1)$ is a polynomial. Its degree, however, is bounded by $d(n - 1) - n = (d - 1)(n - 1) - 1$ for maximum gate degree d — a gate of degree d multiplies up to d column polynomials of degree $< n$, and division by Z_H removes n — which, for the deployed $d = 9$, exceeds the IPA’s degree bound n . The prover therefore splits h into $\lceil (\deg h + 1)/n \rceil$ chunks $h_0, h_1, \dots$ of degree $< n$, with $h(X) = \sum_i X^{ni} h_i(X)$, and commits to each chunk separately; the verifier recombines the chunk *commitments*, weighted by powers of x^n once the evaluation point x is drawn. The prover’s FFTs correspondingly run over an *extended* domain whose size accommodates $\deg C$, a small multiple of n .

Phase 5 — evaluation challenge x . The verifier returns $x \in \mathbb{F}$. The prover evaluates every committed polynomial required by the verifier’s queries, except the quotient chunks, at x , and at the rotated points the constraints reference — ωx for the running products (and for any gate reading an adjacent row), $\omega^{-1}x$ for the permuted lookup input A' — and sends these claimed evaluations as field elements. No evaluation of h is sent: the verifier computes the expected quotient evaluation $C(x)/(x^n - 1)$ from the sent evaluations and enforces it against the collapsed chunk commitment inside the multipoint opening of phases 6–7, where the claimed evaluations themselves also bind to their commitments.

Phase 6 — multipoint opening. The claimed evaluations must now be bound to the commitments, and Halo 2 reduces all the required openings to a single IPA opening. The committed polynomials are grouped by their set of query points: most are queried at $\{x\}$ alone, while the running products are also queried at ωx (and A' at $\omega^{-1}x$), so only a handful of distinct point sets arise. Within each group, a challenge x_1 folds the polynomials into a single combination $q_i(X)$ by powers of x_1 . (The letters q and f here follow the halo2 book’s multipoint-opening notation and are unrelated to the selector and fixed-column polynomials of the Setup paragraph, which are themselves among the polynomials being folded; the index i now ranges over query-point sets, not columns.) For each group, both parties interpolate the low-degree polynomial $r_i(X)$ that agrees with the claimed evaluations on the group’s point set; the quotient $f_i(X) := (q_i(X) - r_i(X))/\prod_z (X - z)$, with z ranging over the group’s points, is a polynomial precisely when the claimed evaluations are correct. The prover combines the f_i by powers of a second challenge x_2 into $f(X)$ and commits to it. The verifier returns a fresh evaluation point x_3 , and the prover sends the evaluation of each q_i at x_3 ; the verifier reconstructs

the expected value of $f(x_3)$ from the $q_i(x_3)$, the $r_i(x_3)$, and the vanishing factors — if each f_i is a genuine polynomial this value matches the committed f , and by Schwartz–Zippel a match at random x_3 convinces the verifier that every claimed evaluation was correct. A final challenge x_4 folds f and the q_i into one polynomial $f^*(X)$, whose expected evaluation at x_3 the verifier assembles from the same data; a single opening of f^* at x_3 then discharges every original opening at once.

Phase 7 — IPA opening. Prover and verifier execute the deployed recursive halving IPA of Remark 5.4 on f^* . The prover sends the blinding commitment S , then $\log_2 n$ pairs (L_j, R_j) , and finally the scalars c, f . The verifier accepts if the resulting deployed-normalisation MSM equation holds; Equation (5) is the symmetric presentation, not the code’s literal equation.

Optional accumulation (recursion; not deployed by Orchard). The next proof’s verifier circuit defers the MSM check $G^{(0)} = \text{Commit}(g(X; u_1, \dots, u_k))$ at the end of phase 7 into an accumulator and folds it with prior accumulators (§5.6). A single decider at the very end validates the entire chain.

The concrete transcript. The deployed implementation instantiates Fiat–Shamir with a transcript built on the BLAKE2b hash function. Every prover message — commitments as curve points, claimed evaluations as field elements — is absorbed into the hash state in the order the protocol fixes, and each verifier challenge is squeezed from the state at its prescribed point, after everything it must depend on has been absorbed. Both parties also absorb the data they share in advance — the vk digest and the unblinded instance commitments — which therefore never appear in the proof. The proof is the ordered sequence of the prover’s messages; the verifier replays the absorptions and recomputes every challenge.

Costs. The prover’s work is dominated by FFTs over the extended domain and by one $O(n)$ -size MSM *per committed polynomial* — several tens in all: each advice column, the permuted lookup columns, the running products, the quotient chunks, and the multipoint polynomial f (typical Orchard Action circuit: $n = 2^{11}$, a few seconds on commodity hardware). The verifier performs $O(\log n)$ field and group operations per proof—recomputing the challenges, the gate-equation field evaluation, and the $\log n$ rounds of phase 7—plus the single deferred $O(n)$ MSM. A batch verifier may combine complete MSM equations using fresh random scalars; that batching is distinct from the optional accumulation scheme of §5.6.

Proof size. Two kinds of data make up a proof. The IPA of phase 7 contributes $2 \log_2 n + 1$ group elements (a blinding commitment and the pairs (L_j, R_j) ; Remark 5.4) plus two final scalars — for Orchard’s $n = 2^{11}$, twenty-two points in the pairs; this is the only part that scales with n , and only logarithmically. Everything else scales with the *circuit structure*, independently of n : one commitment per advice column, two permuted-column commitments and one running product per lookup, the permutation-product and quotient-chunk commitments, the multipoint commitment, and one field element per claimed evaluation. Each Pasta point or field element serialises to 32 bytes. For the Orchard Action circuit the total is fixed by consensus (ZIP 225): a bundle’s proof occupies $2720 + 2272 \cdot a$ bytes for a Actions — just under 5 kB for a single-Action bundle — the per-Action term covering the per-circuit commitments and evaluations, the constant term the evaluations of the verifying key’s fixed-column and permutation polynomials together with the shared quotient, multipoint, and IPA data.

5.8 Instance columns and binding the public statement

Definition 5.1 lists *instance* columns alongside fixed and advice columns, but the arguments of §5.3–§5.7 never single them out. We single them out now, because what an Orchard Action proof *asserts* is precisely a statement about its instance values — the anchor, nullifier nf , net value commitment

cv^{net} , randomised key rk , extracted new-note commitment cmx , the `enableSpend`/`enableOutputs` flags, and—for the post-NU6.3 circuit—the `disableCrossAddress` flag.

The instance polynomial. An instance column is a public-input column: its n entries $\vec{\pi} = (\pi_0, \dots, \pi_{n-1}) \in \mathbb{F}^n$ are not part of the witness but both parties agree on them before verification. Like every other column it is interpolated over H in the Lagrange basis,

$$\pi(X) := \sum_{j=0}^{n-1} \pi_j \ell_j(X) \in \mathbb{F}[X], \quad \deg \pi < n,$$

where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ otherwise (§5.1). The prover writes the public values into a handful of designated rows; all remaining π_j are 0.

Binding by copy constraints. Orchard binds the public values through the permutation argument of §5.3: the instance column participates in the permutation σ , and the circuit equality-constrains each instance cell to the advice cell carrying the corresponding in-circuit value (`halo2`'s `constrain_instance`). A satisfying assignment therefore exists *only* when the advice cells holding the anchor, nf , cv^{net} , rk , cmx , and the applicable `enable/disable` flags equal the public values the verifier supplied. (An alternative design binds public inputs with a dedicated gate $q(X)(a(X) - \pi(X)) = 0$, where the selector q is active exactly on the designated rows; Orchard does not use it — the copy constraint reuses the existing permutation machinery at no extra gate degree.)

Where it enters the protocol. The verifier never takes $\pi(X)$ on trust from the prover. In the deployed IPA backend, *both* parties compute the commitment to each instance polynomial — unblinded, since the values are public, so the two computations agree — and absorb it into the transcript before the first challenge (Phase 1, §5.7); every Fiat–Shamir challenge thereby depends on the public input. Thereafter the instance column is treated like any other: its claimed evaluation $\pi(x)$ appears among the Phase 5 evaluations, enters the gate equation and the permutation expressions, and the multi-point opening of Phase 6 binds it to the instance commitment. Because the verifier computed that commitment itself, from the instance values it intends to verify against, the prover has no freedom over π : an evaluation inconsistent with the verifier's own values fails the opening argument.

Consequence. A prover who targets different public values $\vec{\pi}'$ must open against the instance commitment the *verifier* computes from the genuine $\vec{\pi}$, and must satisfy the copy constraints against those values. Knowledge soundness (§3) then extracts a witness for the relation instantiated at exactly those public values; a prover that does not know such a witness — whatever witnesses it holds for other instances — convinces the verifier with at most negligible probability. This binds the proof to its public statement: an Action proof that a node verifies against an anchor it accepts, a nullifier it will mark spent, and a value commitment it will sum into the net balance check can only verify if the prover knew a witness consistent with *those* values.

5.9 Zero knowledge: hiding the witness in Halo 2

Knowledge soundness (§3) establishes that a convincing prover *knows* a witness; it remains to explain why the proof *reveals* no more than that. Halo 2 is zero-knowledge through two devices — hiding commitments and blinded polynomials — assembled into a simulator.

Hiding commitments. The polynomial commitment of §5.5 is *perfectly hiding*. The prover commits a polynomial p as

$$\text{Commit}(p; r) = \langle \vec{p}, \vec{G} \rangle + [r]H, \quad r \xleftarrow{\$} \mathbb{F}_p,$$

with $\vec{p}$ the coefficient vector of p and H an independent generator of no known discrete-log relation to $\vec{G}$. The $[r]H$ term makes the commitment a *uniform* group element for every p , so a commitment by itself leaks nothing about what the prover committed (the perfect hiding of a Pedersen commitment, *Crypto Guide*). Every commitment the prover sends in phases 1–6 — the advice columns, the permuted lookup columns A' , S' , the permutation and lookup running products, the quotient chunks, and the multipoint polynomial f — has this form. The verifying-key and instance commitments are deliberately unblinded: they commit public data both parties recompute (§5.7, §5.8), so there is nothing to hide.

Blinding the witness polynomials. The prover eventually *opens* a commitment, and an opening reveals evaluations. The prover interpolates the advice columns over the domain H to polynomials of degree $< n$; to prevent those evaluations from leaking the witness, the prover does not use all n rows for circuit data. It reserves a small fixed number of rows at the bottom of the table — the *blinding rows*, never covered by an active gate — and fills them with uniform random field elements, at least one more than the number of times the verifier will open any single column. Its n evaluations pin down a degree- $(< n)$ polynomial, so with enough independent random values planted on the blinding rows the handful the verifier actually queries are jointly uniform and independent of the circuit data. The prover blinds the permutation and lookup running products the same way; Remark 5.2 describes how the constraint system exempts the blinding rows, so the random values never violate an argument. The quotient pieces admit no such treatment — C fully determines them — so their commitment blinders, together with the blinding-row randomness they inherit through C , protect them instead; their single collapsed opening at x is the publicly computable value $C(x)/(x^n - 1)$. (The deployed implementation also commits a uniformly random polynomial alongside the quotient and opens it at the same point, keeping the multipoint argument zero-knowledge.)

Why the openings leak nothing. In each phase the verifier sees only (i) hiding commitments — uniform group elements — and (ii) a bounded number of evaluations of the committed polynomials at the Fiat–Shamir challenge points. The blinding rows make each such evaluation uniformly random subject only to the *public* relations the verifier itself checks; no information about the private witness passes through. The final inner-product opening (§5.5) likewise runs with a blinder, so it too hides its input.

The simulator. Formally, zero knowledge requires a simulator for the *complete joint transcript*, given only the public input. The usual proof samples suitably blinded commitments and evaluations satisfying the verifier equations, then programs fresh Fiat–Shamir queries to the required challenges. Perfect hiding of each commitment is one ingredient, not by itself a proof that this joint distribution matches a real execution: the blinding dimension, transcript order, opening simulation, and freshness of programmed queries must all be covered by the composed protocol theorem. Under those hypotheses the interactive protocol is honest-verifier zero knowledge and its Fiat–Shamir compilation is NIZK in the programmable random-oracle model.

The three properties together. Subject to the full PIOP–PCS composition and Fiat–Shamir hypotheses just stated, Halo 2 targets three guarantees for the relation fixed by a PLONKish circuit: *overwhelming completeness* (an honest prover holding a satisfying witness convinces the verifier except on the negligible transcript degeneracies noted in Remark 5.4), *knowledge soundness* (§3: from any convincing prover an extractor pulls a satisfying witness, so an accepted proof certifies one is *known*), and *zero knowledge* (a witness-free simulator reproduces the proof, so it leaks nothing else). It is exactly this combination — “the prover knows a satisfying witness, and the proof discloses nothing more” — on which the Orchard Action proof rests: knowledge soundness yields balance and nullifier integrity, while zero knowledge yields privacy.

6 Worked example: one computation, carried from claim to conviction

6.1 What we are going to prove, and what “prove” means

Pick a claim small enough to hold in one hand:

“I know a secret x such that $x^3 + x + 5 = 35$.”

The secret is $x = 3$ (indeed $27 + 3 + 5 = 35$). This worked example follows that one sentence until a sceptic is convinced of it — without ever being told that $x = 3$.

Three demands make this nontrivial, and a Halo 2 proof meets all three. The proof has *overwhelming completeness*: an honest prover who knows x convinces the verifier except on negligible transcript degeneracies. It is *sound* (more precisely, *knowledge sound*): a prover who does *not* know a valid x is rejected, except with negligible probability — and “knowledge” is meant literally, in the sense that a satisfying x could in principle be extracted from any prover who succeeds. And it is *zero-knowledge*: the verifier finishes convinced yet learns nothing about x beyond the bare truth of the claim. Completeness is easy; the content of the subject is *soundness with zero-knowledge*, and that is what we will watch being built.

Why this toy? Because it is the Orchard spend statement in miniature. When Alice spends a note she proves “I know a note, a key, and a Merkle path (*Crypto Guide*, §“Merkle trees and commitments to sets”) such that the nullifier — the unique tag whose publication marks the note spent — is computed correctly, the value commitment (a Pedersen commitment to the note’s value; *Crypto Guide*, §“The Pedersen commitment”) opens, and the note sits in the tree” — a conjunction of a few thousand small arithmetic facts over $\mathbb{F}_{p_{\text{Pallas}}}$ in the deployed Orchard Action relation. Our claim is a conjunction of *four* small arithmetic facts. The proof system does not care which; it is the same machine at two scales. Master $x^3 + x + 5 = 35$ and the Action circuit is only larger, not different.

The journey. The route has five legs, and each is a translation that loses nothing:

1. **Computation** → **grid** (§6.2). Rewrite the claim as a table of rows, each row one elementary gate ($\times$ or $+$), with wires between rows forcing shared values.
2. **Grid** → **polynomials** (§6.3). Interpolate each column of the table into a polynomial. “Every row obeys its gate” becomes “one polynomial $G(X)$ is zero at every row.”
3. **Vanishing** → **identity** (§6.4). “ G is zero on all rows” becomes the clean algebraic identity $G(X) = t(X)Z_H(X)$ for some quotient t . A correct computation is a polynomial identity; an incorrect one cannot be.
4. **Identity** → **one random check** (§6.5). The verifier tests the identity at a single random point z . Here lives the magic: a false identity survives a random point with vanishing probability (Schwartz–Zippel).
5. **Trust the openings** (§6.6–§6.9). A binding *commitment* stops the prover from changing her polynomials after seeing z ; the permutation argument enforces the wiring; an instance column pins the public “35”; and blinding makes the whole transcript reveal nothing.

Section 6.12 threads the legs back together into one sentence: why the verifier, having checked a few numbers, is right to be convinced.

We work throughout in the prime field $\mathbb{F}_{97}$. This is a toy size chosen so the reader can check every product by hand. Over Orchard’s roughly 255-bit field, a *fixed* false identity of degree around 2^{14} survives one fresh uniform evaluation challenge with probability around 2^{-240} instead of our visible $3/97$. This is one Schwartz–Zippel term, not the security level of the complete proof system.

6.2 From computation to a grid of constraints

A verifier cannot re-run Alice’s computation: that would need her secret. So we first dismantle the computation into *local* facts, each so simple it involves only a handful of values, and arrange them so that “all the local facts hold, and the values are consistently shared” is equivalent to “the whole computation ran correctly.” This rearrangement is *arithmetisation*; the Halo 2 dialect of it is *PLONKish* (§5.1).

Break the computation into gates. Evaluate $x^3 + x + 5$ the way a machine would, naming each intermediate:

$$x^2 = x \cdot x, \quad x^3 = x^2 \cdot x, \quad s = x^3 + x, \quad s + 5 = 35.$$

Four elementary operations, two multiplications and two additions. Each becomes one row of a table. A row has three *wires* — call them a, b, c , the two inputs and the output — and a bank of *selectors* q_M, q_L, q_R, q_O, q_C that switch on whichever gate this row performs. The universal gate equation, evaluated at every row, is

$$q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0, \quad (6)$$

where PI carries any public input on that row (here, the target 35). A multiplication gate $a \cdot b = c$ is the choice $q_M = 1, q_O = -1$, the rest zero; an addition gate $a + b = c$ is $q_L = q_R = 1, q_O = -1$. Filling in our four operations gives the *execution trace*, Table 2. (The deployed system generalises this fixed selector bank to designer-chosen gates and lookups, declared once and filled row by row at proving time; Section 5.2 shows that machinery on the real Orchard gates.)

row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate performed
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

Table 2: The execution trace for $x = 3$. Each row satisfies the gate equation (6): e.g. row 0 gives $1 \cdot 3 \cdot 3 + (-1) \cdot 9 = 0$, and row 3 gives $1 \cdot 30 + 5 + (-35) = 0$. The selector columns q , and the public input PI are *fixed* (known to the verifier); the wires a, b, c are the prover’s *advice* (her secret trace).

Wire the rows together: copy constraints. The table as printed has a fatal loophole. Nothing yet forces the x in row 0 to be the same x in rows 1 and 2, nor the output $x^2 = 9$ of row 0 to be the input of row 1. A cheating prover could put $x = 3$ in row 0 but $x = 8$ in row 2 and satisfy each gate *in isolation* while computing nonsense overall. We must assert that certain cells are *equal*. These are the *copy constraints* (or *equality constraints*):

$$\underbrace{a_0 = b_0 = b_1 = b_2}_{\text{all are } x=3}, \quad \underbrace{c_0 = a_1}_{x^2=9}, \quad \underbrace{c_1 = a_2}_{x^3=27}, \quad \underbrace{c_2 = a_3}_{s=30}.$$

They are what turn a list of unrelated gates into a single dataflow. Enforcing them is a small argument in its own right (§6.7); for now simply note what they say.

What the grid achieves. The claim is now equivalent to a purely combinatorial assertion: *there exist values for the advice cells a, b, c such that (i) every row satisfies the gate equation (6) with the fixed selectors, and (ii) every copy constraint holds*. Knowing x is exactly knowing how to fill the advice columns. The verifier knows the fixed columns and the public input; the prover must convince him that satisfying advice exists, while revealing none of it. Everything from here is machinery for doing precisely that.

6.3 From the grid to polynomials

The grid has a finite number of rows, and we want to make statements about “all rows at once” that a verifier can check by looking at *one* place. Polynomials are the tool: a low-degree polynomial is rigid — pinned down everywhere by its behaviour at a few points — and that rigidity is what we will weaponise.

An index set for the rows. Our table has $n = 4$ rows. Choose four distinct field elements to name them. The inspired choice is the n -th roots of unity: a value ω with $\omega^4 = 1$ and no smaller power equal to 1. In $\mathbb{F}_{97}$ we may take $\omega = 22$, for which $\omega^2 = 96 = -1$ and $\omega^4 = 1$, so the row labels are

$$H = \{\omega^0, \omega^1, \omega^2, \omega^3\} = \{1, 22, 96, 75\} = \{1, 22, -1, -22\}.$$

Row i now lives at the point ω^i . (The roots of unity are chosen not for elegance but for speed: they make the polynomial arithmetic below an FFT (*Math Guide*, §“The fast Fourier transform”). Any 4 distinct points would do for correctness.)

Columns become polynomials. Each column of the table is four numbers, one per row. *Interpolate*: for the advice column a there is a unique polynomial $a(X)$ of degree < 4 with

$$a(\omega^0) = 3, \quad a(\omega^1) = 9, \quad a(\omega^2) = 27, \quad a(\omega^3) = 30,$$

and likewise $b(X), c(X)$, and the (publicly known) selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI , each a polynomial in X of degree < 4 . The column *is* the polynomial, sampled at the rows. A polynomial of degree < 4 is exactly four coefficients, so no information is gained or lost — only re-encoded into a form that bends to algebra.

All gates at once, as one polynomial. Now assemble the left-hand side of the gate equation (6), but with the *polynomials* in place of the per-row numbers:

$$G(X) := q_M(X) a(X) b(X) + q_L(X) a(X) + q_R(X) b(X) + q_O(X) c(X) + q_C(X) + PI(X). \quad (7)$$

This G has degree at most 9 (the term $q_M ab$ multiplies three degree-3 polynomials). Here is the point of the whole construction. Evaluate G at the label of row i : because each polynomial returns that row’s value at ω^i , $G(\omega^i)$ is *exactly* the gate equation (6) for row i . Therefore

$$G(\omega^i) = 0 \text{ for every row } i \iff \text{every gate is satisfied.}$$

For our honest trace this is true by construction: G vanishes at all four points of H . “The computation is correct” has become “ G vanishes on H .” We have replaced four separate checks with a single statement about one polynomial — and it is a statement we can now make checkable in one shot.

6.4 From vanishing to a single identity

“ G vanishes on the set H ” is still a statement about four points. One more step compresses it into a statement with *no* reference to the individual rows at all — a bare algebraic identity — and this is the technical heart of arithmetic proving.

The vanishing polynomial. The polynomial that is zero exactly on H , and nowhere else, and as simply as possible, is

$$Z_H(X) := \prod_{i=0}^3 (X - \omega^i) = X^4 - 1.$$

(The product of $(X - \zeta)$ over all n -th roots of unity ζ telescopes to $X^n - 1$; that is the only reason roots of unity are convenient here.) The elementary fact we lean on is the factor theorem, applied n times:

$$G \text{ vanishes on all of } H \iff (X - \omega^i) \mid G \text{ for each } i \iff Z_H(X) \mid G(X). \quad (8)$$

Divisibility is the same as the existence of a quotient. So:

$$\text{every gate holds} \iff G(X) = t(X) Z_H(X) \text{ for some polynomial } t(X).$$

A correct computation is *precisely* the existence of this quotient t . For our honest trace the division comes out even: G has degree 9, Z_H has degree 4, and

$$t(X) = \frac{G(X)}{X^4 - 1}$$

is an honest polynomial of degree 5 — the remainder is zero, as it must be.

Why a cheat cannot produce t . Suppose the prover's advice is *wrong* — some gate fails. Then G does *not* vanish on all of H , so by (8) Z_H does *not* divide G , so *no* polynomial t satisfies $G = t Z_H$. The dividing line is exact: the quotient exists if and only if the trace is valid. This is the lever the verifier will pull. Concretely, take a prover who claims $x^2 = 10$ and writes $c_0 = 10$ in row 0 (Table 2) instead of 9. Now row 0's gate reads $3 \cdot 3 - 10 = -1 \neq 0$, so the tampered G^* has $G^*(\omega^0) = -1 = 96$ while still vanishing at the other three rows. Dividing G^* by Z_H leaves a nonzero remainder; the best the cheater can do is publish the quotient $t^* = \lfloor G^*/Z_H \rfloor$, and then

$$D(X) := G^*(X) - t^*(X) Z_H(X) = 24(1 + X + X^2 + X^3)$$

is a *nonzero* polynomial of degree 3. The identity she needs is false, and D measures exactly by how much. Keep D in view: the next section is about catching it cheaply.

6.5 Why checking one random point is a proof

The prover wishes to convince the verifier that $G(X) = t(X) Z_H(X)$ as polynomials, with G built from her secret advice. The verifier will not read the polynomials — they encode the witness, and there are too many coefficients to be cheap anyway. Instead:

1. The prover *commits* to her polynomials a, b, c and the quotient t (Section 6.6 says what a commitment is and why it binds her); the selector and public-input polynomials the verifier already knows.
2. The verifier picks a point $z \in \mathbb{F}$ *uniformly at random* and sends it.
3. The prover *opens* her commitments at z : she returns the field elements $a(z), b(z), c(z), t(z)$ together with short proofs that these are the true evaluations of the committed polynomials.
4. The verifier forms $G(z)$ from the openings and the (self-evaluated) public polynomials via (7), computes $Z_H(z) = z^4 - 1$, and checks the *single field equation*

$$G(z) \stackrel{?}{=} t(z) Z_H(z). \quad (9)$$

He accepts if and only if (9) holds. That is the entire test: one equation between numbers.

The honest case (completeness). If the prover is honest then $G = t Z_H$ holds as polynomials, hence at every point, hence at z . With our trace and $z = 20$:

$$Z_H(20) = 20^4 - 1 = 46, \quad t(20) = 67, \quad G(20) = 75 = 67 \cdot 46 = t(20) Z_H(20). \quad \checkmark$$

The verifier accepts, every time, for every z .

The dishonest case (soundness), and the one idea that makes it work. Suppose the trace is invalid — some gate fails, say at row i , so $G^*(\omega^i) \neq 0$ and G^* does *not* vanish on all of H . Take *any* polynomial t^* the cheating prover might commit to (the honest quotient, or anything else) and set

$$D(X) := G^*(X) - t^*(X) Z_H(X).$$

Were D the zero polynomial, then $G^* = t^* Z_H$ would be divisible by Z_H and so vanish on all of H — which it does not. Hence D is a *nonzero* polynomial, however cleverly t^* was chosen. And D is fixed *before* z is drawn: committing to a, b, c pins down the polynomial G^* (the verifier never receives G^* itself — from the openings at z he reconstructs only the single *number* $G^*(z)$), and committing to t^* pins down the rest, so the entire polynomial D is frozen by the commitments before the challenge is revealed. The check (9) passes at z exactly when $D(z) = 0$ — that is, exactly when the random z happens to be a *root* of D . And here is the lever: *a nonzero polynomial of degree d has at most d roots* (Schwartz–Zippel, in one variable just the factor theorem). A degree- d polynomial is pinned down by any $d + 1$ of its values, so it has no freedom to vanish at more than d points without collapsing to zero everywhere; a nonzero low-degree polynomial is therefore *mostly* nonzero, its roots a sparse set that a random probe almost surely misses. So

$$\Pr_z[\text{cheat accepted}] = \Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|}.$$

A liar is caught with overwhelming probability over the verifier’s single coin toss. *This* is what “proving” means here: not that the verifier re-checks the work, but that a false statement is an algebraic object so rigid it cannot hide — it disagrees with the truth almost everywhere, and one random probe almost surely lands where it disagrees.

Watch it happen. For our $x^2 = 10$ cheat, $D(X) = 24(1 + X + X^2 + X^3)$ has degree 3, so at most 3 roots — and in $\mathbb{F}_{97}$ its roots are exactly $\{22, 96, 75\}$, the three rows where the tampered gate still held. The cheat therefore slips past only if the verifier’s random z lands in that three-element set:

$$\Pr[\text{cheat accepted}] = \frac{3}{97}.$$

At the verifier’s actual draw $z = 20 \notin \{22, 96, 75\}$, we get $G^*(20) = 20$ but $t^*(20) Z_H(20) = 64$, and $20 \neq 64$: caught. The fraction $3/97$ is large only because our field is a toy. Over Orchard’s roughly 255-bit field, a fixed nonzero gap polynomial has at most $\deg D$ roots. For $\deg D$ around 2^{14} , one independently uniform z contributes a Schwartz–Zippel term around 2^{-240} . The complete proof has other randomized identities and cryptographic assumptions, and Fiat–Shamir grinding multiplies a per-draw term by the adversary’s query budget. In particular this isolated figure is not an overall 240-bit security claim; the Pasta-group discrete-log layer has about 126 bits of classical concrete security.

Bounding the degree. The argument needs D to have *low* degree relative to $|\mathbb{F}|$ — otherwise “ $d/|\mathbb{F}|$ roots” is no constraint. This is why a circuit’s gates are kept to low degree and why t , of degree ≤ 5 here (and up to a few times n in general), is split into bounded pieces in the deployed system. Low degree is not an efficiency footnote; it is the source of soundness.

6.6 Why the prover cannot wriggle: binding commitments

The random-point argument has an obvious gap: it assumed the prover’s polynomials were *fixed before* she saw z . If she could choose a, b, c, t *after* learning z , she would simply pick numbers making (9) hold and reveal nothing real. A *polynomial commitment* closes exactly this gap.

What a commitment must do. A commitment to a polynomial f is a short string $C = \text{Commit}(f)$ with two properties (the *Crypto Guide* develops both):

- **Binding.** Having published C , the prover cannot later open it as two different polynomials; C pins down f . So committing first, then receiving z , then opening, leaves her no freedom: she is answering for the *one* polynomial she fixed in advance — the situation Section 6.5 assumed.
- **Hiding.** C reveals nothing about f itself. (This is what will let the proof be zero-knowledge, §6.9.)

Add an *evaluation proof*: given C and a point z , the prover can prove “ $f(z) = v$ ” for the committed f without exposing f . With binding plus trustworthy evaluation proofs, the four-step protocol of Section 6.5 is exactly as sound as its idealised version: the opened numbers $a(z), \dots, t(z)$ are forced to be the true evaluations of fixed polynomials, so the check (9) tests a genuine, pre-committed identity at a genuinely random point.

How Halo 2 commits, in one breath. Halo 2 uses the *inner-product argument* (IPA). Encode f by its coefficient vector $\vec{f} = (f_0, \dots, f_m)$ and fix a public list of independent curve points $\vec{G} = (G_0, \dots, G_m)$ with no known relationship. The commitment is the single curve point

$$C = \langle \vec{f}, \vec{G} \rangle = \sum_i f_i G_i,$$

a *Pedersen* vector commitment: binding because finding two openings would solve a discrete-log relation among the G_i , and hiding once a random blinding term is added. To prove $f(z) = v$ is to prove a statement about the inner product of $\vec{f}$ with the vector $(1, z, z^2, \dots)$, and the IPA does this in a logarithmic number of rounds, each *halving* the vectors, until the claim is trivial — so the evaluation proof is $O(\log m)$ group elements and, decisively, needs *no trusted setup* (§6.11 says why, and what the fold’s one expensive step costs). The full fold and its soundness are developed in §5.5. For our purposes one property is enough: *the commitment binds*, so the prover is honestly answering for polynomials she could not change once z was on the table.

6.7 Enforcing the wiring: the permutation argument

One promise from Section 6.2 is still unpaid: the copy constraints. The gate identity of Sections 6.3–6.5 checks each row *in isolation*; it never says that a_0, b_0, b_1, b_2 are the same value x , or that row 1’s input is row 0’s output. Without that, a prover could satisfy every gate with an incoherent dataflow. The *permutation argument* adds the missing constraint, and it does so with the same philosophy: turn “these cells are equal” into “a certain product is 1,” then check the product at a random point.

Equalities as a permutation. Group the cells into classes that must share a value — here $\{a_0, b_0, b_1, b_2\}$, $\{c_0, a_1\}$, $\{c_1, a_2\}$, $\{c_2, a_3\}$ — and let σ be the permutation that cycles each displayed class in the order written. Give every cell a distinct *label*: cell in row j of the a -, b -, c -column gets $\omega^j, k_1\omega^j, k_2\omega^j$ respectively, where k_1, k_2 place the three columns in disjoint cosets so no two labels collide (in $\mathbb{F}_{97}$ we may take $k_1 = 2, k_2 = 3$). Asserting “cells in a class are equal” is then equivalent to asserting that the map sending each cell’s *value-with-label* to its *value-with- σ -permuted-label* is consistent — a balance between two multisets.

The grand product. With verifier challenges β, γ , the prover builds a running product whose end-point enforces

$$\prod_{\text{cells } (i,j)} (v_i(\omega^j) + \beta(\text{label}) + \gamma) = \prod_{\text{cells } (i,j)} (v_i(\omega^j) + \beta(\sigma\text{-label}) + \gamma).$$

If the wiring is honest, numerator and denominator run over the *same* multiset of factors, so the equality holds identically. If a copy constraint is false, the resulting nonzero polynomial in β, γ can

still vanish for a particular challenge pair, but only with probability $O(n/|\mathbb{F}|)$ by Schwartz–Zippel. Thus the converse is probabilistic, not an “if and only if” for fixed challenges. The actual circuit enforces cross-multiplied row updates, not literal field divisions, so a zero factor does not make the constraint undefined. The prover commits to the running-product polynomial and the verifier checks its boundary and row-to-row update at random z .

A two-cell glimpse. Strip the product down to a single wired pair, $c_0 = a_1$ (both should be 9), carrying labels L_{c_0}, L_{a_1} that σ swaps. Their factors satisfy

$$(9 + \beta L_{c_0} + \gamma)(9 + \beta L_{a_1} + \gamma) = (9 + \beta L_{a_1} + \gamma)(9 + \beta L_{c_0} + \gamma),$$

because the shared value 9 makes the two numerator factors a mere reordering of the two denominator factors — the labels may permute under σ precisely because the values agree. Now let the prover cheat the copy, $a_1 = 8 \neq 9$: the factor $8 + \beta L_{a_1} + \gamma$ on one side generally has no match on the other, and the two products differ. Equal values let labels permute; one unequal value strands a factor, and the whole product feels it.

Watch it happen. For our honest trace, with $\beta = \gamma = 2$, the grand product comes out to exactly 1. Tamper one wire — set $b_1 = 4$, breaking the copy $b_1 = x = 3$ — and the same product becomes $61 \neq 1$: the inconsistent dataflow is exposed without any value being revealed.

6.8 Proving a value sits in a table: lookups

Gates handle algebra; some constraints are not algebra. “This chunk indexes an entry of the Sinsemilla generator table” or “this cell is a ten-bit number” has no low-degree formula. Written as one polynomial gate it would need degree about the size of the table — which in our toy field would eat the soundness budget outright (§6.5), and at any scale balloons the quotient polynomial the prover must split, commit, and open (§6.4). Yet Orchard’s circuit needs such range and table facts constantly.

The statement has a familiar shape. “Every entry of my column A appears somewhere in the table S ” is, like the copy constraints, a statement about *multisets*. One wrinkle: membership is not a permutation — A may repeat one table entry and ignore the rest — so the prover first tidies both sides. She commits to A' , her column reshuffled so equal values sit in contiguous runs, and S' , a reshuffle of the table aligned so that each run in A' begins beside its matching table value. Two *row-local* checks then pin membership: each row of A' either *starts a run* ($A'_i = S'_i$: the value is vouched for by the table) or *continues one* ($A'_i = A'_{i-1}$: equal to the row above, already vouched for). What certifies the two reshuffles is the §6.7 machinery with one deliberate change: *no labels*. Labelled factors pinned one particular wiring σ ; here the reshuffles are the prover’s own choice, so a single running product with unlabelled factors — row by row, $(A' + \beta)(S' + \gamma)$ against $(A + \beta)(S + \gamma)$ — certifies exactly that each primed column is *some* reshuffle of its original, which is all membership needs. Induction up the rows does the rest.

On the toy’s own four-row grid: constrain cells to two-bit values with the table $S = (0, 1, 2, 3)$ and the column $A = (2, 3, 2, 2)$. Grouped, $A' = (2, 2, 2, 3)$; align $S' = (2, 0, 1, 3)$, a reshuffle of S placing 2 beside the first run and 3 beside the second. Row 0 starts a run ($2 = 2$), rows 1 and 2 continue it (2 above), row 3 starts a run ($3 = 3$) — every check row-local, every polynomial low-degree, however large the table. A forged $A = (2, 7, 2, 2)$ dies: its 7 lands at the start of a run in A' , and no reshuffle of a table without a 7 can put a 7 beside it.

This is how the deployed circuit affords its ten-bit range checks and its 2^{10} -entry Sinsemilla generator table: a table fact costs a few committed columns and one more running product, and the gates stay low-degree. The deployed declarations of both are exhibited in §5.2; the exact constraints, the boundary row, the padding of short columns, and multi-column tables are developed in §5.4.

6.9 Binding the public statement, and hiding the witness

Two finishing pieces complete the picture, one for soundness and one for secrecy.

Pinning the public “35”: the instance column. A proof must attest to *this* statement, not some other the prover finds convenient. The target 35 is public, so it is not advice but *instance* data: it sits in the public-input polynomial $PI(X)$ of (7), which the verifier builds himself (it is -35 on row 3 and 0 elsewhere). Because PI enters the very identity being checked, a prover who tried to prove “ $x^3 + x + 5 = 36$ ” would be checking a *different* G and her honest trace would no longer vanish on H . The public input is thus welded into the same polynomial identity that soundness already guards (§5.8). The verifier proves something about the claim he chose, not one the prover swapped in.

Revealing nothing: zero-knowledge. So far the prover has handed over commitments and a few evaluations $a(z), b(z), \dots$ at a random z . Do these leak the witness? They could: an evaluation is a linear equation in the secret coefficients, and enough of them would pin x down. Halo 2 closes this the standard way — *blinding*. Each committed polynomial is padded with a few extra random coefficients (equivalently, the trace reserves a few random rows), and the commitment carries an independent random blinding term. The gate and permutation arguments are arranged to ignore those random rows (Remark 5.2), so they do not disturb soundness; in the full zero-knowledge argument, the openings at z are masked by fresh randomness and their joint distribution reveals nothing about x . Formally one exhibits a *simulator* that, under the composition hypotheses of §5.9, matches the transcript distribution knowing only that the statement is true. The simulator discussion appears in §5.9; the intuition is simply: *commitments hide, and blinded openings are noise pinned to one honest constraint*.

6.10 Removing the verifier: the Fiat–Shamir transform

One awkwardness remains. Everything above needs a *live* verifier for exactly one service: once the commitments a challenge tests are pinned, somebody the prover cannot predict must choose that challenge — z here, β and γ for the permutation argument, the coins inside each opening round (§6.6). Nothing else about him matters: his messages are coin tosses, and his final act is a computation anyone could run. A payment protocol cannot work this way. Alice would have to sit in a live session with every full node that will ever validate her transaction.

The fix is almost impudent. Wherever the verifier would send a fresh challenge, the prover instead computes it herself as

$$\text{challenge} := H(\text{everything sent so far}),$$

the digest of the entire conversation to that point, public statement and every commitment included, under a fixed hash function H . The ordering that §6.6 fought for is now enforced by construction: a commitment is an *input* of the hash, so it is fixed before the challenge that tests it exists at all.

Why is a hash as good as a coin? Because the prover’s only route to a favourable challenge is to vary some input of H — and every input she controls is something the finished proof must stand behind: the statement she is claiming, or a commitment she must open. Each variation re-rolls the challenge blindly: one hash evaluation buys one ticket in a lottery she wins only with the relevant per-draw soundness error. If that error is ϵ and she makes at most Q distinct transcript queries, a union bound gives at most $Q\epsilon$ for this grinding route; for the illustrative identity check, ϵ is around 2^{-240} . Thus the query budget must be included rather than declared irrelevant. Two fine points matter. The hash must digest the *whole* transcript — leave the statement or any commitment out, and she aims precisely at what the hash cannot see. And the guarantee is proved with H modelled as a random oracle, a strong but standard heuristic; the *Crypto Guide* (§“The Fiat–Shamir transform: from interactive to non-interactive”) gives the theorem and its caveats.

What this buys is the “N” in SNARK: *non-interactive*. The proof becomes a single static object — commitments and openings, nothing else — and verification becomes recomputing the challenges

by the same hash and running the same checks. No session, no coordination: a full node that has never met Alice verifies her Action years later, exactly as the deployed system does (§5.7 describes the transcript it uses).

6.11 The “Halo” in Halo 2: deferring the expensive check

One cost has stayed quietly non-trivial. The opening proofs of §6.6 shrink by halving, so the verifier’s rounds are logarithmic — except at the very end, where checking the final fold means recomputing one combination of *all* the generators $G_0, \dots, G_m$: a wall of group arithmetic proportional to the whole circuit’s size. When checking a batch, the deployed library random-combines the complete per-proof MSM equations and evaluates one combined MSM. That is a standard randomized batch check, not the recursive accumulation operation of §5.6. Where the cost would be truly ruinous is *recursion*. For a circuit to verify a *proof* — the step that would let one proof attest to a whole chain of others — everything the verifier does must be re-expressed as gates, and a circuit-sized wall of group arithmetic inside a circuit defeats the point.

For optional recursion, Halo’s namesake trick is to defer payment. The expensive claim has a special shape — “this point equals that known combination of the generators” — and two claims of this shape can be *folded* into one by the move this worked example keeps meeting: commit, draw a challenge, take a random combination, a few group operations in place of a wall. So the recursive verifier checks everything cheap, and instead of paying the expensive step it folds the claim into a running *accumulator* carried along the chain. One final check, run once at the chain’s end and outside any circuit, pays a single wall of group arithmetic for the entire history. The bookkeeping’s soundness reduces to the discrete logarithm, exactly like the commitments it manages (§5.6); the cycle of curves that keeps the recursive arithmetic in the right fields is developed in §4.3.

A last remark completes the picture begun in §6.6: none of this ever needed a ceremony. The generators $G_0, \dots, G_m$ are nothing-up-my-sleeve points, hashed into existence, with no secret behind them that anyone ever knew — nothing to leak, nothing to destroy. The pairing-based KZG family buys constant-size opening proofs and constant-time verification at the price of a *trusted setup*: secret randomness contributed through a ceremony, whose compromise permits silent forgeries (the *Crypto Guide*, §“Polynomial commitment schemes”, constructs that scheme and the forgery its trapdoor permits). Zcash’s earlier shielded pools likewise needed parameter ceremonies for their Groth16 proof systems, though that is a separate construction. Halo 2’s inner-product commitment trades proof size for exactly this freedom; being rid of ceremonies was a stated reason Orchard chose it.

6.12 Why the verifier is convinced

Stand back and read the chain in the direction the verifier experiences it. He accepts a proof. What has he thereby learned?

1. The check $G(z) = t(z)Z_H(z)$ passed at his random z . Because the commitment *binds* (§6.6), the polynomials behind the openings were fixed before z ; because z was random and a false identity disagrees with the true one at all but $\leq \deg D$ points (§6.5), passing supports the conclusion that $G(X) = t(X)Z_H(X)$ holds as polynomials, except with the corresponding $\deg D/|\mathbb{F}|$ Schwartz–Zippel error for this fixed check.
2. $G = tZ_H$ means $Z_H \mid G$, which means G vanishes on all of H (§6.4), which means *every gate equation holds at every row* (§6.3).
3. The permutation grand product passed (§6.7), so, except with its random-challenge soundness error, the *copy constraints* hold too: the per-row gates are wired into one coherent computation, not four disconnected coincidences.
4. The public-input polynomial fixed the target at 35 (§6.9), so the computation he has verified is the one he asked about.

Putting these together — and recalling from §6.2 that filling the advice columns is knowing x , since those cells are the wires carrying x and its powers — there exists an assignment to the advice columns, a value of x , making the entire arithmetisation of “ $x^3 + x + 5 = 35$ ” hold. Under the composed PIOP, PCS-extraction, and Fiat–Shamir hypotheses of §3, an extractor for the *full* protocol can obtain such an x . The IPA opening extractor alone establishes knowledge of committed polynomial data; it does not by itself prove that those polynomials encode a satisfying circuit witness. And by zero-knowledge (§6.9) the verifier learns no more than this single bit of truth. The verifier has checked a small transcript and a logarithmic-size opening proof; its confidence is governed by the sum of all algebraic error terms and the PCS, Fiat–Shamir, and discrete-log assumptions, not by the isolated 2^{-240} term alone.

The same machine, at Orchard scale. Nothing above used that our statement was small. Replace the four gates by the few thousand constraints of the Orchard Action — nullifier integrity, the value-commitment opening, the Merkle path recomputation, and key re-randomisation (*Crypto Guide*, §“Key re-randomisation and unlinkability”) — and the columns grow from four rows to 2^k , the gate polynomial gains custom high-degree terms, and a Sinsemilla lookup table (§6.8; the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”) joins the permutation. But the spine is identical: arithmetise into a grid, interpolate to polynomials, compress “all rows correct” into one identity $G = t Z_H$, commit, and test at a random point. When a Zcash full node verifies a shielded spend, randomized polynomial-identity tests of the kind in Section 6.5 form its algebraic core. The full verifier also checks permutation and lookup relations, multiple openings, transcript challenges, and the final IPA equation before concluding that the asserted Action relation holds.

What the formal treatment supplies. The earlier sections discharge every deferral above: the inner-product argument and why its openings are trustworthy (§5.5); the algebraic-group-model extractor behind “the prover knows x ” (§3); the soundness of the accumulation fold sketched in §6.11 (§5.6); and the cycle of curves that makes recursion efficient (§4.3). This worked example has supplied the picture those formalisms formalise: *how random evaluations of committed polynomials contribute to knowing that an unseen computation was done right.*